

# XDR Analyst Blueprint

Rafina

June 2026

## 1. Alerting and Detection Processes Alerting and Detection Processes (23 %)

Este dominio evalúa si entiendes cómo Cortex XDR detecta actividad sospechosa o maliciosa y cómo convierte esa actividad en alertas e incidentes investigables.

Cortex XDR **recibe datos de distintas fuentes, analiza** esos datos, **genera alertas** cuando detecta algo sospechoso, **prioriza** esas alertas y las **agrupa** en incidentes para que el analista pueda investigarlas.

Es decir, este bloque cubre la parte inicial del ciclo SOC:

datos → detección → alerta → priorización → agrupación → incidente

No estamos todavía en la respuesta al incidente ni en la investigación profunda. Eso pertenece más al dominio 2. Aquí lo importante es **entender cómo nace una alerta**, por qué se considera relevante, cómo se ordena por prioridad y cómo acaba formando parte de un incidente.

En Cortex XDR, una **alerta puede originarse por muchas razones**: comportamiento sospechoso en un endpoint, detección de malware, una regla BIOC, un IOC, una alerta analítica, una regla de correlación, actividad de red, información de identidad o datos procedentes de fuentes externas. Por eso el examen no se limita a preguntarte “qué es una alerta”, sino que puede preguntarte de dónde viene, qué motor la genera, cómo se clasifica y qué hace Cortex XDR con ella después.

La diferencia conceptual más importante de este bloque es:

- **Evento**: dato bruto o actividad registrada.
- **Alerta**: evento o conjunto de eventos que Cortex XDR considera sospechoso.
- **Incidente**: agrupación de alertas relacionadas que representan una posible actividad maliciosa o una investigación común.

**Ejemplo sencillo:**

Un usuario ejecuta un archivo sospechoso. Ese archivo lanza PowerShell. PowerShell intenta conectarse a una IP externa rara. Cortex XDR puede generar varias alertas: una por el archivo, otra por el comportamiento, otra por la conexión, otra por coincidencia con un IOC. En vez de dejar todas esas alertas sueltas, Cortex XDR intenta agruparlas dentro de un mismo incidente para que el analista vea la historia completa.

Por tanto, este dominio no trata solo de “ver alertas”, sino de entender el mecanismo de detección y organización de esas alertas.

### 1.1. Identify and explain different types of alerts and alert sources

Este punto evalúa si sabes **reconocer de dónde viene una alerta y qué lógica la ha generado**.

En Cortex XDR, una alerta no aparece “porque sí”. Siempre **hay una fuente de datos y un mecanismo de detección detrás**. Cortex XDR consolida las alertas no informativas procedentes de sus fuentes de detección para que el analista pueda triarlas e investigarlas desde la tabla de alertas.

**La idea básica es:**

fuentes de datos → lógica de detección → alerta → posible agrupación en incidente

**Por ejemplo:**

1. Un endpoint ejecuta un archivo sospechoso.
2. El agente de Cortex XDR analiza el archivo.
3. WildFire o Local Analysis lo consideran malicioso.
4. Cortex XDR genera una alerta.
5. Esa alerta puede acabar agrupada con otras dentro de un incidente.

Existen dos conceptos clave:

- **Alert source:** Origen de la alerta o del dato que la genera
- **Alert type:** Tipo de detección o lógica que ha producido la alerta

**Ejemplo:** una alerta puede tener como **source** el agente de Cortex XDR, pero el tipo puede estar relacionado con malware, WildFire, Local Analysis, exploit prevention, etc.

Cortex XDR **genera alertas desde varias categorías principales**: reglas configuradas por el usuario, como BIOC, IOC o correlation rules; agents; firewalls; Analytics; e integrations de terceros.

Aquí se mostraran las fuentes principales de alerta:

- El **Cortex XDR Agent** es una de las fuentes más importantes. Está instalado en endpoints y servidores, y puede generar alertas por actividad sospechosa o maliciosa observada en el host.

**Definición:** Agent alert = alerta generada desde el endpoint.

**El examen puede plantearte una situación como:** “Un archivo desconocido se ejecuta en un endpoint y posteriormente WildFire lo clasifica como malware. ¿Qué fuente de alerta interviene?”

La **respuesta** sería: Cortex XDR Agent / WildFire-related alert, dependiendo de cómo esté formulada la opción.

**WildFire** es relevante porque el agente puede consultar la reputación/veredicto de un archivo. Si no hay veredicto previo, Cortex XDR puede enviar la muestra para análisis en WildFire. Si WildFire identifica el archivo como malware, puede bloquearlo.

También existe **Local Analysis**, que permite al agente emitir un veredicto local usando machine learning o reglas cuando el archivo es desconocido o cuando WildFire no está disponible todavía.

- Un **IOC** (Indicator of Compromise) es un indicador estático y conocido. Por ejemplo, un hash SHA256 malicioso, una IP maliciosa, un dominio sospechoso o un nombre de archivo.

**Ejemplo de examen:** “Quieres detectar si aparece en tu entorno un hash SHA256 asociado a ransomware. ¿Qué tipo de regla utilizarías?”

**Respuesta:** IOC rule.

- Un **BIOC** (Behavioral Indicator of Compromise) no busca necesariamente un indicador fijo, sino un comportamiento sospechoso. Por ejemplo, una PowerShell ejecutada con parámetros ofuscados o un proceso de Office lanzando cmd.exe.

**Ejemplo de examen:** “Quieres generar una alerta cuando Microsoft Word lance PowerShell con argumentos sospechosos. ¿Qué tipo de regla deberías usar?”

**Respuesta:** BIOC rule.

- Las **Analytics alerts** las genera el motor analítico de Cortex XDR cuando detecta actividad sospechosa (de red o no) que se desvía de un comportamiento base.

Aquí ya no hablamos solo de una regla manual creada por el administrador. El motor de Analytics analiza eventos, establece un baseline de comportamiento y genera alertas cuando detecta desviaciones relevantes. Palo Alto indica que el motor de Analytics genera una alerta cuando detecta actividad sospechosa compuesta por múltiples eventos que se desvía del baseline establecido con el tiempo.

**Ejemplos:**

- Usuario que inicia sesión desde una ubicación inusual.
  - Host que empieza a comunicarse con destinos anómalos.
  - Aumento anormal de conexiones.
- **Analytics BIOC / ABIOC:** está relacionado con Analytics, pero mantiene la idea de detección comportamental. Según la documentación, si Cortex XDR Analytics está habilitado, Cortex XDR puede usar Analytics BIOC para establecer comportamiento base y detectar desviaciones.

**Para examen:**

- BIOC normal = regla de comportamiento definida.
  - Analytics BIOC / ABIOC = comportamiento/anomalía generado o gestionado en el contexto de Analytics.
- Las **correlation rules** generan alertas correlando múltiples eventos, normalmente mediante lógica XQL. Palo Alto indica que las correlation rules ayudan a analizar la relación entre múltiples eventos de múltiples fuentes usando el motor basado en Cortex Query Language, XQL.

**Ejemplo:** Una correlation rule puede unir esos eventos y generar una alerta.

- Una autenticación sospechosa en Okta.
- Después, acceso desde una IP anómala.
- Después, descarga masiva de datos.
- Después, ejecución de una acción administrativa.

**Clave de examen:** Correlation rule = relación entre varios eventos, normalmente de varias fuentes, usando lógica XQL. Si el enunciado habla de correlacionar varios eventos en una ventana temporal, o de usar una consulta para detectar un patrón, piensa en correlation rule.

- **Firewall / network alerts:** Cortex XDR también puede generar alertas a partir de datos de red, especialmente cuando ingiere datos de productos como Next-Generation Firewall, Panorama, Prisma Access o Strata Logging Service.

Desde datos de NGFW o Strata Logging Service, Cortex XDR puede generar alertas de tipo Analytics, Correlation Rules, IOC y BIOC cuando sea relevante. Una limitación importante es que Analytics, IOC y BIOC requieren logs normalizados, mientras que las Correlation Rules pueden levantarse sobre logs normalizados y no normalizados.

- **Third-party / external alerts:** Cortex XDR puede ingerir alertas externas para enriquecer la investigación. La documentación indica que Cortex XDR puede ingerir alertas desde cualquier fuente externa, unirlas con datos relevantes de endpoint y mostrarlas en incidentes, tablas de alertas y vistas de causality.

| Tipo de alerta         | Qué la genera                      | Qué detecta                                       | Ejemplo de examen                                |
|------------------------|------------------------------------|---------------------------------------------------|--------------------------------------------------|
| XDR Agent alert        | Agente en endpoint                 | Malware, exploit, comportamiento local sospechoso | Archivo malicioso ejecutado en un endpoint       |
| IOC alert              | Regla IOC                          | Indicador estático conocido                       | Hash SHA256 malicioso detectado                  |
| BIOC alert             | Regla BIOC                         | Comportamiento sospechoso                         | Word lanza PowerShell                            |
| Analytics alert        | Motor Analytics                    | Desviación respecto a baseline                    | Usuario con comportamiento anómalo               |
| Analytics BIOC / ABIOC | Analytics                          | Comportamiento anómalo basado en baseline         | Actividad que se desvía del patrón normal        |
| Correlation rule alert | Regla de correlación / XQL         | Relación entre varios eventos                     | Login sospechoso + descarga masiva               |
| Firewall/network alert | NGFW, Panorama, Prisma Access, SLS | Amenazas o anomalías de red                       | Conexión a dominio C2                            |
| Third-party alert      | Integración externa/API/SIEM       | Alertas de otras herramientas                     | Alerta de phishing recibida desde email security |

Figura 1: Resumen

**BIOC vs Analytics:** BIOC suele ser una regla de comportamiento definida. Analytics se basa en baseline, desviación o análisis de múltiples eventos. **Ejemplo:** Regla definida para detectar cmd.exe lanzado por Office → BIOC. Usuario que se comporta de forma anómala respecto a su histórico → Analytics.

**Analytics vs Correlation Rule:** Analytics detecta anomalías o patrones mediante el motor analítico. Correlation rule aplica una lógica definida, normalmente XQL, para relacionar eventos. **Ejemplo:**

- Host con patrón de tráfico anómalo respecto a su baseline → Analytics.
- Consulta XQL que genera alerta si un usuario falla 10 logins y luego consigue login exitoso desde la misma IP → Correlation rule.

#### Frase clave:

En Cortex XDR, las **alertas pueden originarse** en agentes, firewalls, Analytics, reglas o integraciones; el **tipo de alerta depende de la lógica que la genera**: IOC para indicadores estáticos, BIOC para comportamientos sospechosos, Analytics para desviaciones de baseline, correlation rules para correlación de eventos y third-party alerts para alertas externas ingeridas.

## 1.2. Explain the alert prioritization handling process

La **idea central** es esta: No todas las alertas tienen la misma prioridad. Cortex XDR permite priorizar usando puntuaciones, estrellas, campos destacados y reglas personalizadas según el contexto de la organización.

El **alert prioritization handling process** es el proceso por el cual un analista SOC ordena, filtra y atiende primero las alertas o incidentes más importantes.

En Cortex XDR, la priorización no depende de un único campo. Normalmente combina varios elementos:

- **Severity:** indica gravedad técnica de la alerta o incidente
- **Incident score:** da una puntuación numérica para comparar incidentes
- **Starred alerts/incidents:** marca alertas o incidentes importantes para seguimiento
- **Featured fields:** resalta hosts, usuarios, IPs o atributos críticos
- **Custom rules:** ajustan la prioridad según el contexto de la organización

#### **Ejemplo sencillo:**

Una alerta de severidad media en un portátil de laboratorio puede ser menos prioritaria que una alerta de severidad media en un controlador de dominio. Por eso Cortex XDR permite usar reglas basadas en activos, usuarios, IPs o grupos para elevar la prioridad de aquello que realmente importa en la organización.

#### **1.2.1. Incident scoring**

El **incident scoring** es un mecanismo de priorización que asigna una puntuación numérica a un incidente. Sirve para comparar incidentes entre sí y decidir cuáles deben investigarse antes.

Palo Alto indica que Cortex XDR ofrece **incident starring** e **incident scoring** para ayudar a priorizar incidentes. El scoring asigna un valor numérico a cada incidente para reflejar su severidad, y permite comparar scores, filtrar incidentes marcados y priorizar los más urgentes. Además, el incident scoring está disponible en Cortex XDR Pro.

Cortex XDR contempla **tres formas principales de scoring**:

1. En el **rule-based scoring**, se crean reglas que asignan puntuaciones a alertas que cumplan ciertos criterios. Esas reglas pueden basarse en atributos como hostnames, IP addresses, users, Active Directory groups, Azure groups u organizational units. Si una alerta coincide con una o varias reglas, Cortex XDR asigna los scores correspondientes; si coinciden varias reglas, el score de la alerta se agrega.

#### **Ejemplo:**

Regla: “Si la alerta es de tipo XDR BIOC y tiene severity Critical, asignar score 30.”

Entonces, cuando aparezca una alerta que cumpla esa condición, Cortex XDR le aplica esa puntuación. El ejemplo de la documentación usa precisamente una regla con Score = 30 y filtro Alert Source = XDR BIOC AND Severity = Critical

El incident score no sale de la nada. Cortex XDR calcula el score del incidente a partir del total combinado de los scores de las alertas incluidas

en el incidente. También se puede ver un desglose del score desde el panel de detalles.

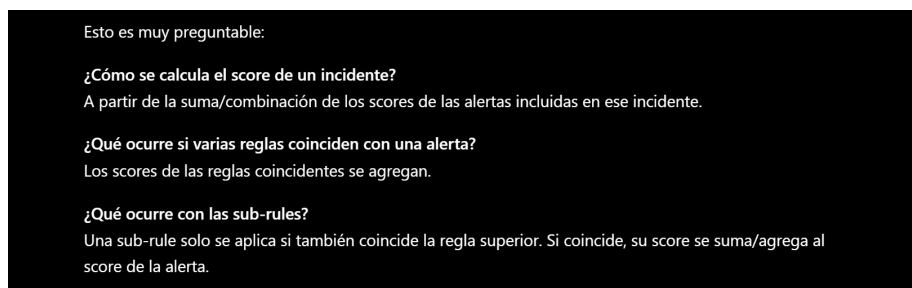


Figura 2: Preguntas interesantes.

2. **SmartScore:** Es un scoring automático basado en machine learning, análisis estadístico, atributos del incidente e información comparativa/cross-customer. Según la documentación, puede tardar hasta 48 horas en calcular y mostrar score tras su primera activación si necesita suficiente información.
3. **Manual scoring:** Cuando Cortex XDR no asigna un score, o el analista quiere sobrescribirlo, se puede definir manualmente una puntuación para el incidente.

Para examen, quédate con esta jerarquía:

- Rule-based score: lo configuras tú con reglas.
- SmartScore: lo calcula Cortex XDR automáticamente.
- Manual score: lo fija el usuario/analista.

No confundir el **incident score** con **severity**. Un incidente puede tener una severidad determinada, pero además puede tener un score numérico que ayuda a priorizarlo.

### 1.2.2. Alert starring

La lógica práctica es que se pueden destacar incidentes importantes y también crear configuraciones que automaticen el marcado cuando una alerta cumple ciertos atributos.

**Starring** significa marcar con una **estrella** los elementos que el analista o la organización considera importantes.

Cortex XDR permite marcar incidentes con una estrella para centrarse en los más relevantes. Los incidentes marcados se pueden filtrar en la **página de incidentes** y en el **dashboard** de gestión de incidentes. Cortex XDR identifica estos incidentes con una estrella morada.

**La idea de examen:** Starring = mecanismo visual y operativo para destacar y filtrar incidentes/alertas importantes.

Hay dos formas:

1. **Manual starring:** el analista marca manualmente un incidente.
2. **Automatic starring configuration:** Cortex XDR marca automáticamente incidentes que contienen alertas con atributos definidos. Si una alerta coincide con esos atributos, se marca la alerta y también el incidente que la contiene. Las configuraciones se gestionan en Incident Response → Incident Configuration → Starred Alerts.

**Ejemplo:**

Quieres que cualquier alerta que afecte a un servidor crítico, un usuario privilegiado o una IP sensible aparezca destacada. Creas una configuración de starring para esos atributos. Si aparece una alerta que coincide, Cortex XDR la marca con estrella y también marca el incidente asociado.

“Un incidente está starred. ¿Eso implica automáticamente que su severidad es Critical?”

**Respuesta: No.** Significa que ha sido marcado como importante manualmente o por una configuración, pero no equivale necesariamente a una severidad crítica.

### 1.2.3. Featured fields

Los **featured fields** son campos o valores destacados que quieres vigilar especialmente dentro de las alertas.

Cortex XDR **permite etiquetar atributos concretos de alertas como featured alert fields** para ayudar a rastrear alertas que involucren **hosts, usuarios e IPs concretas**. Las alertas que contienen un **valor coincidente** se identifican con una **bandera** en el campo **Alert Name** de la tabla de alertas. Después, esos campos destacados pueden usarse para filtrar la tabla de alertas y para crear reglas de incident scoring.

**La idea de examen:** Featured field = atributo importante que se quiere resaltar en alertas, por ejemplo host, usuario o IP crítica.

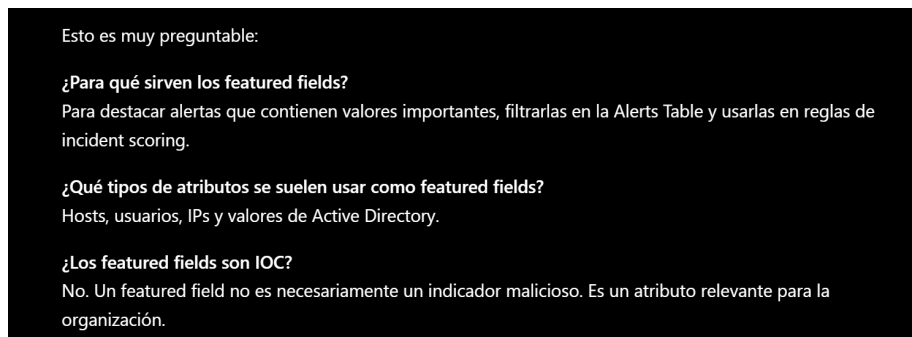


Figura 3: Preguntas interesantes.

**No confundas featured fields con IOCs.** Un IOC es un indicador de compromiso: hash, IP, dominio, etc. Un featured field es un campo que la organización quiere destacar: por ejemplo, un servidor crítico o un usuario sensible.

Una misma IP podría ser IOC si es maliciosa, o featured field si es una IP crítica de la organización. El significado depende del uso.

#### 1.2.4. Custom prioritization configuration

La **custom prioritization configuration** es la parte que permite adaptar la priorización al contexto real de la empresa.

La priorización por defecto puede no ser suficiente. Una alerta de severidad media puede ser urgente si afecta a un activo crítico, y una alerta alta puede ser menos urgente si afecta a un entorno de pruebas. Por eso Cortex XDR permite personalizar la priorización usando principalmente:

- Configuración: para qué sirve
- Scoring rules: asignar puntos a alertas según condiciones
- Starring configurations: marcar automáticamente alertas/incidentes relevantes
- Featured fields: resaltar hosts, usuarios, IPs o atributos críticos
- Manual scoring: ajustar la puntuación cuando sea necesario

En Cortex XDR, las reglas de scoring se configuran desde Incident Response → Incident Configuration → Scoring Rules. En una **scoring rule** defines un nombre, una puntuación, si es regla raíz o sub-rule, si aplica solo a la primera alerta del incidente, y los filtros/atributos de alerta que deben coincidir.

Un punto concreto que puede caer:

**Apply score only to first alert of incident**

Esta opción hace que el score se aplique solo a la primera alerta del incidente que coincida con la regla. Las alertas posteriores del mismo incidente no reciben puntuación adicional por esa misma regla. Por defecto, el score se aplica solo a la primera alerta que coincide con la regla y sub-rule.

Esto es importante porque evita inflar artificialmente el score de un incidente cuando se repite muchas veces la misma condición.

#### **Ejemplo:**

Tienes una regla que asigna +20 puntos si una alerta afecta a un controlador de dominio. Si el incidente contiene 30 alertas repetidas sobre el mismo controlador, quizá no quieres sumar +20 treinta veces. Con “Apply score only to first alert of incident”, solo se suma una vez.

#### **Preguntas tipo test probables**

**1. ¿Qué permite incident scoring en Cortex XDR?**

Asignar una puntuación numérica a incidentes para priorizarlos.

**2. ¿Qué métodos de scoring existen?**

Rule-based scoring, SmartScore y manual scoring.

**3. ¿Cómo calcula Cortex XDR el score de un incidente?**

Combinando los scores de las alertas incluidas en el incidente.

**4. ¿Para qué sirve starring?**

Para marcar y filtrar incidentes/alertas considerados importantes.

**5. ¿Un incidente starred es necesariamente Critical?**

No. Starred indica destacado/priorizado, no necesariamente severidad crítica.

**6. ¿Qué son featured fields?**

Atributos destacados de alertas, como hosts, usuarios o IPs relevantes.

**7. ¿Para qué se pueden usar los featured fields?**

Para filtrar alertas y crear reglas de incident scoring.

**8. ¿Qué evita “Apply score only to first alert of incident”?**

Que un mismo criterio sume puntuación repetidamente dentro del mismo incidente.

**9. ¿Qué diferencia hay entre detection rule y scoring rule?**

La detection rule genera alertas; la scoring rule prioriza alertas/incidentes ya generados.

**10. ¿SmartScore es manual?**

No. SmartScore se calcula automáticamente mediante machine learning y análisis estadístico.

Figura 4: Preguntas interesantes.

### **1.3. Explain the incident creation process**

Cortex XDR **no quiere** que el analista trabaje solo con alertas sueltas. Cuando una alerta es suficientemente relevante, intenta **agruparla** con otras alertas, activos y artefactos relacionados para formar un incidente investigable.

Una **Alerta** es una notificación de actividad sospechosa.

Un **incidente** es un contenedor de investigación. Palo Alto lo define como un objeto que agrupa alertas relacionadas, activos y artefactos que proceden de una misma causa raíz. Esa causa raíz puede ser un ataque autocontenido, una combinación de malware y exploits, o una secuencia de actividad relacionada.

Un incidente puede contener:

- **Alerts:** alertas que notifican actividad sospechosa o eventos relevantes
- **Assets:** endpoints, hosts o usuarios afectados
- **Artifacts:** ficheros, procesos, dominios, IPs, firmas, hashes u otros atributos implicados

#### ¿Cuándo se genera un incidente? El proceso básico es:

Se genera una alerta → Cortex XDR comprueba si encaja con un incidente existente → si encaja, la añade → si no encaja, crea un nuevo incidente.

La documentación lo expresa así: un incidente puede contener una o más alertas relacionadas; las alertas se enlazan a incidentes haciendo coincidir su contenido. Si una alerta nueva no coincide con ningún incidente existente, Cortex XDR crea un nuevo incidente. Cuando la alerta se enlaza a un incidente, también se enlazan sus assets y artifacts asociados.

Para examen, **memoriza esta frase:** Una alerta crea un nuevo incidente solo si es relevante para crear incidente y no coincide con ningún incidente existente.

#### Ejemplo:

Aparece una alerta porque un endpoint ejecuta un binario malicioso. Cortex XDR mira si ya existe un incidente relacionado con ese host, ese proceso, ese hash, esa cadena de causality o esos artefactos. **Si existe**, añade la alerta al incidente. **Si no existe**, abre un nuevo incidente.

**No todas las alertas crean incidentes** En general, solo las alertas de severidad Medium, High o Critical pueden crear un nuevo incidente. Las alertas Informational normalmente se incluyen como Insights dentro de incidentes, y existen algunas excepciones en detectores de Analytics que pueden crear incidentes a partir de alertas Low. Además, las alertas Low generadas por correlation rules no se agrupan en incidentes.

En **correlation rules** hay una regla concreta: si una alerta generada por una correlation rule tiene severidad Medium o superior, se abre automáticamente un incidente; si es Low, no se agrupa en incidentes.

**Qué ocurre cuando una alerta se añade a un incidente?** Cuando Cortex XDR decide que una alerta pertenece a un incidente existente, no añade solo la alerta. También añade su contexto.

Es decir, se actualiza el incidente con:

- La nueva alerta.

- Los hosts afectados.
- Los usuarios relacionados.
- Los artefactos implicados.
- La severidad/desglose de alertas.
- Las fuentes de alerta.

En la **tabla de incidentes**, Cortex XDR muestra campos como Alerts Breakdown, número de alertas por severidad, hosts afectados, usuarios, fuentes del incidente, hora de creación, hora de última actualización y tácticas/técnicas MITRE asociadas.

**Un detalle útil:** la descripción del incidente puede generarse a partir del nombre de la primera alerta añadida, el host y usuario afectados, o el número de usuarios y hosts afectados.

**Esto puede caer como idea conceptual:** La primera alerta puede influir en la descripción inicial del incidente, pero el incidente puede crecer después con más alertas, assets y artifacts.

#### **Límites de agrupación de alertas en un incidente**

Cortex XDR no mantiene un incidente abierto eternamente aceptando alertas nuevas. Usa umbrales para mantener los incidentes frescos y manejables.

Un incidente deja de aceptar nuevas alertas relacionadas cuando se cumple alguno de estos límites:

- 30 días desde la creación del incidente: pasado ese tiempo, las nuevas alertas relacionadas irán a otro incidente
- 14 días desde la última alerta detectada: si no hay nuevas alertas durante ese periodo, deja de agrupar en ese incidente
- 1.000 alertas en el incidente: si alcanza ese límite, las siguientes alertas relacionadas van a un nuevo incidente

La documentación indica que, cuando se alcanza un umbral, Cortex XDR deja de aceptar alertas en ese incidente y agrupa las alertas relacionadas posteriores en un nuevo incidente. También puedes ver el estado en el campo **Alerts Grouping Status** de la tabla de incidentes.

#### **¿Qué ocurre con incidentes resueltos o auto-resueltos?**

Cuando todas las alertas asociadas a un incidente están **marcadas** como **resueltas**, el incidente se pone automáticamente como **Auto-Resolved**. Además, cualquier actualización hecha sobre una alerta afecta al incidente asociado.

Pero hay un detalle importante: Cortex XDR puede seguir agrupando alertas en un incidente Auto-Resolved durante un **máximo de 6 horas**. Si durante ese periodo aparece una alerta que coincide con el incidente, Cortex XDR lo reabre. Pasadas esas 6 horas, las nuevas alertas relacionadas se enlazan a un nuevo incidente para una nueva investigación.

Pregunta tipo: “Un incidente está Auto-Resolved y aparece una alerta coincidente 3 horas después. ¿Qué puede hacer Cortex XDR?” Respuesta: Puede reabrir el incidente y agrupar la alerta en él.

“¿Y si aparece 8 horas después?” Respuesta: Se vincularía a un nuevo incidente para una nueva investigación.

### **¿Qué papel tienen las exclusiones en la creación de incidentes?**

Las **alert exclusions** afectan directamente a la creación y composición de incidentes.

Si creas una regla de exclusión de alertas, Cortex XDR oculta futuras alertas que coincidan con esos criterios y las excluye de incidentes y resultados de búsqueda. Si un incidente contiene solo alertas excluidas, Cortex XDR cambia el estado del incidente a Resolved - False Positive y puede enviar una notificación al assignee.

### **Relación con causality y matching de contenido**

No entro todavía en profundidad porque el alert grouping y data stitching van en el punto 1.4, pero para entender la creación de incidentes tienes que conocer la idea básica: Cortex XDR agrupa alertas por contenido relacionado.

Uno de los mecanismos usados es la relación por Causality Instance ID / CID. Las alertas dentro del mismo CID pueden formar parte de una misma cadena de ejecución, y eso es uno de los métodos que Cortex XDR usa para agrupar alertas en un incidente.

#### **Relación con alertas externas**

Cortex XDR también puede ingerir alertas externas y mostrarlas en incidentes relevantes. En el caso de fuentes externas, Cortex puede unir esas alertas con datos relevantes de endpoint y mostrar los assets y artifacts relacionados en vistas de causality.

#### **La idea de examen:**

Un incidente no tiene por qué estar formado solo por alertas nativas del agente XDR. También puede incluir alertas externas si han sido ingeridas y correlacionadas correctamente.

## 10. Esquema del proceso completo

Este es el flujo que debes memorizar:

### 1. Se genera una alerta.

Puede venir del agente, Analytics, BIOC, IOC, correlation rule, firewall, fuente externa, etc.

### 2. Cortex XDR evalúa su severidad y relevancia.

Normalmente Medium, High y Critical pueden crear incidentes. Low e Informational tienen limitaciones.

### 3. Cortex XDR comprueba si coincide con un incidente existente.

Busca relación por contenido, assets, artifacts, causality, fuentes o contexto.

### 4. Si coincide, la añade al incidente existente.

También añade assets y artifacts asociados.

### 5. Si no coincide, crea un nuevo incidente.

Ese incidente será una nueva unidad de investigación.

### 6. El incidente se actualiza con nuevas alertas relacionadas.

Hasta que alcance límites de agrupación: 30 días, 14 días sin nuevas alertas o 1.000 alertas.

### 7. Si todas sus alertas se resuelven, puede pasar a Auto-Resolved.

Pero puede reabrirse durante 6 horas si aparece una alerta coincidente.

Figura 5: Workflow de una alerta

**Qué severidades suelen crear incidentes:** Medium, High y Critical.

**Qué pasa con Informational:** normalmente se incluyen como Insights.

**Qué pasa con Low:** normalmente no crean incidente, aunque hay excepciones; las Low de correlation rules no se agrupan en incidentes.

**Qué pasa cuando una alerta se añade a un incidente:** se añaden también sus assets y artifacts asociados.

**Qué efecto tienen las alert exclusions:** ocultan futuras alertas coincidentes y las excluyen de incidentes y búsquedas.

## 1.4. Explain the concepts of alert grouping and data stitching

**La idea general es:** **Alert grouping** agrupa alertas relacionadas en incidentes. **Data stitching** une datos de distintas fuentes para dar contexto y reconstruir la historia del ataque.

### 1.4.1. Alert grouping

**Alert grouping** es el proceso por el cual Cortex XDR agrupa alertas relacionadas dentro de un mismo incidente.

En vez de crear un incidente independiente por cada alerta, Cortex XDR intenta determinar si varias alertas forman parte de la misma actividad, la misma causa raíz o la misma cadena de ataque. La documentación define un

incidente como un contenedor que agrupa alertas, assets y artifacts relacionados que proceden de una misma causa raíz.

El objetivo es reducir ruido y mejorar el triage. Si cada alerta se tratara de forma independiente, el analista perdería contexto. Al agruparlas, Cortex XDR permite ver el ataque como una historia completa.

**Ejemplo simple:**

- Un usuario abre un documento malicioso.
- Ese documento lanza powershell.exe.
- PowerShell descarga un payload.
- El payload intenta conectarse a un dominio externo.
- WildFire detecta el archivo como malware.
- Un BIOC detecta el comportamiento de PowerShell.
- Una alerta de red detecta la conexión sospechosa.

Cortex XDR puede agrupar esas alertas en un único incidente porque forman parte de una misma actividad.

**La idea de examen es esta:**

Alert grouping = varias alertas relacionadas → un único incidente investigable.  
Cortex XDR puede agrupar alertas por relación entre distintos elementos:

- Mismo endpoint / host: varias alertas en el mismo equipo
- Mismo usuario: actividad sospechosa asociada al mismo usuario
- Mismos artifacts: hash, IP, dominio, proceso, fichero
- Misma cadena de causality: procesos relacionados dentro de una misma ejecución
- Misma actividad de red: conexiones relacionadas entre origen y destino
- Misma secuencia temporal: alertas ocurridas dentro de una ventana relevante
- Misma causa raíz: varias señales generadas por un mismo ataque

No hace falta que en el examen recuerdes un algoritmo interno exacto. Lo importante es entender el principio: **Cortex XDR agrupa alertas cuando detecta relación contextual entre ellas.**

La documentación también explica que el **Causality Analysis Engine** correlaciona actividad de los sensores de detección para establecer cadenas de causality, identifica artefactos relevantes en cada alerta y agrega alertas relacionadas con un evento dentro de un incidente.

Esto es importante: **la causality ayuda al grouping.**

#### 1.4.2. Relación con causality

La **causality chain** representa la secuencia de procesos, eventos y acciones que explican cómo se produjo la alerta.

En Cortex XDR, la vista de causality muestra una cadena de causality, información general, forensics highlights y una tabla de eventos. Además, permite seleccionar nodos de la cadena para ver detalles y ejecutar acciones sobre procesos concretos.

Para este punto, quédate con esto:

- Alert grouping responde a “qué alertas van juntas”.
- Causality responde a “cómo están relacionadas esas alertas dentro de una cadena de actividad”.

#### 1.4.3. Data stitching / log stitching

En la documentación de Cortex XDR se suele llamar **log stitching**, aunque el blueprint usa el término más general **data stitching**. La idea es la misma: correlacionar datos de distintas fuentes para construir una visión completa de la actividad.

Palo Alto explica que, para proporcionar una imagen completa de los eventos, Cortex XDR correlaciona firewall network logs, endpoint raw data y cloud data a través de los sensores de detección. Ese proceso de correlacionar logs de distintas fuentes se denomina log stitching y ayuda a identificar origen y destino de procesos de seguridad y conexiones de red.

**Dicho de forma simple:**

Data stitching = unir piezas de datos dispersas para reconstruir contexto.

Cortex XDR puede hacer stitching con datos como:

- Endpoint raw data: procesos, ficheros, registro, ejecución, actividad local
- Firewall/network logs: origen, destino, puertos, conexiones, amenazas de red
- Cloud data: actividad en servicios cloud
- SaaS logs: actividad en aplicaciones SaaS
- Identity logs: autenticaciones, usuarios, comportamiento de identidad
- Third-party logs/alerts: datos procedentes de herramientas externas

La documentación sobre fuentes externas indica que Cortex XDR puede **ofrecer visibilidad** sobre datos externos mediante **log stitching**, datos raw consultables con XQL, alertas reportadas por proveedores en la tabla de alertas/incidentes/vistas, y alertas generadas por Cortex XDR sobre esos logs, como analytics alerts.

Esto es muy importante para el examen: Cortex XDR no se limita a endpoints. El valor de XDR es correlacionar endpoint, red, identidad, cloud y terceros.

Cortex XDR no solo guarda datos brutos. También puede unir logs y eventos en esquemas comunes llamados **stories**, disponibles mediante presets de Cortex XDR. La documentación de datasets indica que Cortex XDR “stitches” logs y eventos en common schemas llamados **stories** para investigar eventos de forma más eficiente.

La idea para memorizar:

**Stories** = datos ya correlacionados/normalizados que representan actividad común y facilitan investigación y consultas.

**Ejemplo:** Una network story puede unir información de conexión de distintas fuentes para representar mejor una comunicación entre origen y destino.

#### 1.4.4. Conclusión

- Alert grouping / Agrupa alertas relacionadas / Crea o alimenta un incidente
- Data/log stitching / Correlaciona datos de distintas fuentes / Construye contexto, stories y relaciones
- Causality / Reconstruye la cadena de eventos o procesos / Explica causa raíz y secuencia
- Deduplication / Consolida eventos o alertas repetidas / Reduce duplicados

La documentación de **alert deduplication** indica que la deduplicación agrupa eventos de seguridad idénticos que ocurren en el mismo endpoint dentro de una ventana temporal, consolidándolos en una sola alerta accionable. También señala que se aplica estrictamente a alertas cuyo alert\_name contiene WildFire o Local Analysis; otros tipos se procesan individualmente y no se deduplican.

## 15. Trampas típicas

**Trampa 1: "Alert grouping y data stitching son lo mismo."**

No. Grouping agrupa alertas; stitching correlaciona datos.

**Trampa 2: "Alert grouping solo agrupa alertas idénticas."**

No. Agrupa alertas relacionadas. Las idénticas/repetidas pertenecen más al concepto de deduplication.

**Trampa 3: "Data stitching solo usa datos de endpoint."**

No. Puede correlacionar endpoint, firewall, cloud, SaaS, identidad y fuentes externas.

**Trampa 4: "Causality es lo mismo que timeline."**

No. Causality explica relaciones causa-efecto entre procesos/eventos. Timeline muestra una secuencia cronológica.

**Trampa 5: "Un incidente acepta alertas relacionadas sin límite."**

No. Hay límites: 30 días desde creación, 14 días desde última alerta o 1.000 alertas.

**Trampa 6: "Deduplication es equivalente a grouping."**

No. Deduplication reduce repetidos; grouping construye incidentes con alertas relacionadas.

**Trampa 7: "Las alertas externas no pueden participar en stitching."**

No. Cortex XDR puede recibir logs o alertas externas y aportar visibilidad mediante stitching, XQL, incidentes y alertas.

Cortex Centro d...

Figura 6: Resumen

## 2. Incident Handling and Response (34 %)

Este apartado evalúa si sabes investigar la evidencia de una alerta. Ya no estamos en “cómo se crea una alerta”, sino en qué hace el analista cuando tiene una alerta o incidente delante.

### La idea general es:

Cortex XDR te da evidencias para entender qué ocurrió, cómo ocurrió, qué activos están afectados, qué usuario está implicado, cuál fue la causa raíz y qué alcance tuvo la actividad.

### 2.1. Review and investigate alert evidence

El apartado *Review and investigate alert evidence* se centra en la capacidad del analista para revisar e investigar la evidencia asociada a una alerta o incidente en Cortex XDR. En esta fase, el objetivo no es únicamente observar el nombre de la alerta o su severidad, sino comprender el contexto completo de la actividad detectada.

Cuando Cortex XDR genera una alerta, esta puede estar asociada a múltiples evidencias: endpoint afectado, usuario implicado, procesos ejecutados, ficheros creados o modificados, conexiones de red, indicadores de compromiso, cadena de causalidad, línea temporal y artefactos forenses. El analista debe utilizar esta información para determinar qué ocurrió, cómo ocurrió, qué activos están afectados, cuál fue la causa raíz y qué acciones de respuesta son necesarias.

De forma general, la investigación de una alerta debe responder a las siguientes preguntas:

- ¿Qué actividad sospechosa o maliciosa se ha detectado?
- ¿En qué endpoint, usuario o activo se ha producido?
- ¿Qué proceso o evento originó la alerta?
- ¿Existen procesos hijos, conexiones de red o ficheros relacionados?
- ¿La actividad forma parte de una cadena de ataque más amplia?
- ¿Hay evidencias de compromiso de identidad?
- ¿Cuál es el alcance y el impacto potencial del incidente?

Para responder a estas preguntas, Cortex XDR proporciona varias capacidades clave: *Forensics*, *Identity Threat Detection and Response* (ITDR), *Causality Chain* y *Timeline*.

#### 2.1.1. Forensics

La funcionalidad de **Forensics** permite recoger y analizar evidencias procedentes de endpoints afectados. Su finalidad es proporcionar al analista una visión más profunda de lo que ha ocurrido en el sistema investigado.

A diferencia de una simple revisión de la alerta, el análisis forense permite obtener información adicional sobre artefactos, ficheros, registros, eventos del sistema, actividad de procesos y otros elementos relevantes para reconstruir el incidente.

En Cortex XDR, una investigación forense puede incluir:

- **Collection:** recogida de artefactos desde uno o varios endpoints.
- **Triage:** recogida enfocada para obtener datos relevantes rápidamente
- Análisis de ficheros sospechosos.
- Revisión de procesos ejecutados.
- Análisis de cambios en el registro.
- Revisión de logs de eventos.
- Identificación de artefactos relevantes.
- **Investigation timeline:** construcción de una línea temporal de investigación.
- **Alerts table:** alertas detectadas dentro de la investigación

Un concepto importante dentro de Forensics es el **triage**. Una colección de triage permite recoger información relevante de un endpoint de forma rápida para apoyar la investigación inicial. Puede incluir artefactos forenses, rutas de archivos definidas por el usuario, listados de archivos, logs de eventos y hives del registro. Existe una limitación, un máximo de diez endpoints por colección de triage.

**Hunt collections** permiten buscar actividad sospechosa en endpoints para ayudar a resolver una investigación. Se usan cuando quieres buscar una actividad específica en un gran número de hosts, por ejemplo saber qué endpoints ejecutaron un malware, qué usuarios accedieron a un archivo concreto o a qué endpoints accedió un usuario determinado.

**La diferencia principal es:**

**Triage** = quiero recoger mucha evidencia de uno o pocos endpoints para entender qué pasó. **Hunt** = quiero buscar una evidencia concreta en muchos endpoints para saber dónde más ocurrió.

La idea importante para el examen es que **Forensics se utiliza para recoger y analizar evidencias del endpoint con el objetivo de determinar la causa raíz, el alcance y el impacto de un incidente.**

No debe confundirse Forensics con la *Causality Chain*. La cadena de causalidad muestra relaciones causa-efecto entre procesos y eventos, mientras que Forensics permite recoger y analizar evidencias más amplias del endpoint.

### 2.1.2. Identity Threat Detection and Response (ITDR)

**Identity Threat Detection and Response (ITDR)** se centra en la detección e investigación de amenazas relacionadas con la identidad. En muchos ataques modernos, el compromiso no se produce únicamente mediante malware, sino mediante el uso indebido de cuentas válidas, credenciales robadas o abuso de privilegios.

La documentación de Palo Alto describe ITDR como un **add-on** que habilita funciones como Asset Roles Configuration, Advanced Analytics Alert layout, Risk Management Dashboard, User/Host Risk View, Designated Analytics for Compromised Accounts e Insider Threat Coverage.

ITDR permite analizar el riesgo asociado a usuarios, cuentas, hosts y roles dentro de la organización. Es especialmente relevante para investigar:

- Cuentas comprometidas.
- Abuso de privilegios.
- Actividad anómala de usuarios.
- Movimiento lateral mediante credenciales válidas.
- Amenazas internas o *insider threats*.
- Accesos inusuales a recursos críticos.

En una investigación, ITDR aporta contexto adicional sobre el usuario o activo implicado. Por ejemplo, una alerta puede no parecer crítica si se analiza únicamente desde el punto de vista del endpoint, pero puede adquirir mayor relevancia si el usuario afectado pertenece a un grupo privilegiado, accede a sistemas sensibles o presenta un comportamiento anómalo respecto a su actividad habitual.

Algunas capacidades asociadas a ITDR son:

- **User Risk View**: vista de riesgo asociada a usuarios.
- **Host Risk View**: vista de riesgo asociada a hosts.
- **Asset Roles Configuration**: configuración de roles de activos y usuarios.
- **Compromised Account Analytics**: detección de posibles cuentas comprometidas.
- **Insider Threat Coverage**: cobertura frente a amenazas internas.
- **Advanced Analytics Alert layout**: contexto adicional para alertas analíticas.

La idea clave para el examen es que **ITDR no se centra principalmente en malware, sino en identidad, usuarios, cuentas, roles, riesgo y comportamiento anómalo relacionado con credenciales o privilegios.**

Por tanto, si una pregunta describe actividad sospechosa asociada a una cuenta válida, accesos anómalos, abuso de privilegios o comportamiento extraño de un usuario, el enfoque correcto suele estar relacionado con ITDR.

#### **Ejemplo de investigación ITDR**

Imagina esta alerta: Un usuario inicia sesión desde una ubicación inusual. Después accede a un host crítico. Luego ejecuta acciones administrativas. Más tarde descarga muchos datos.

Puede que no haya malware. Pero puede haber una cuenta comprometida.

### **2.1.3. Causality Chain**

La **Causality Chain** es una de las capacidades más importantes de Cortex XDR para investigar alertas. Representa la relación causa-efecto entre procesos, eventos, alertas e insights relacionados con una actividad sospechosa.

Su objetivo es mostrar cómo se llegó a una alerta. No se limita a ordenar eventos cronológicamente, sino que muestra qué proceso originó una acción, qué procesos hijos se ejecutaron, qué ficheros se crearon o modificaron y qué conexiones de red se generaron.

Un ejemplo típico de cadena de causalidad sería:

```
winword.exe
-> powershell.exe
-> descarga de payload
-> ejecución de payload.exe
-> conexión a IP externa
-> generación de alerta
```

En este caso, la alerta no debe analizarse de forma aislada. El analista debe revisar el proceso padre, los procesos hijos, los comandos ejecutados, los ficheros asociados, las conexiones de red y los artefactos relacionados.

La **Causality view** es la vista interactiva donde investigas esa cadena. Palo Alto indica que muestra la causa y efecto de eventos de toda la cadena de ejecución de procesos que llevó a la alerta. También proporciona insights accionables sobre la alerta y los procesos relacionados.

La **Causality Chain** permite responder preguntas como:

- ¿Qué proceso originó la actividad sospechosa?
- ¿Qué proceso fue el padre de la alerta?
- ¿Qué procesos hijos se ejecutaron?
- ¿Qué comandos se lanzaron?
- ¿Qué ficheros, hashes, IPs o dominios están relacionados?

- ¿La actividad forma parte de una cadena de ataque más amplia?

Un concepto asociado es la **Causality Instance** (CI), que representa el alcance concreto de una cadena de causalidad asociada a una alerta o actividad. La CI permite agrupar eventos, procesos y alertas que pertenecen a una misma cadena de ejecución. **CI** = instancia concreta de causality asociada a una alerta o actividad.

El **Causality Analysis Engine** correlaciona actividad de sensores de detección para establecer cadenas de causality que identifican la causa raíz de cada alerta. También identifica una timeline forense de eventos, ayuda a determinar alcance y daño, identifica artefactos relevantes y agrega alertas relacionadas dentro de incidentes.

Para el examen, la idea clave es que **la Causality Chain muestra la relación causa-efecto que llevó a la generación de una alerta y ayuda a identificar la causa raíz de la actividad sospechosa.**

Es importante no confundir la *Causality Chain* con la *Timeline*. La primera explica relaciones causa-efecto; la segunda muestra el orden temporal de los eventos.

#### 2.1.4. Timeline

La *Timeline* muestra la secuencia temporal de eventos, alertas y actividad relacionada con una investigación. Su función principal es ayudar al analista a reconstruir el ataque en orden cronológico.

Mientras que la *Causality Chain* responde a la pregunta ‘¿cómo están relacionados los eventos?’, la *Timeline* responde a la pregunta ‘¿en qué orden ocurrieron los eventos?’.

También **se diferencia explícitamente timeline y causality view**: la causality view muestra eventos y procesos relacionados que Cortex XDR considera importantes, mientras que la Timeline muestra todos los eventos, alertas, informational BIOC y correlation rules relacionados a lo largo del tiempo.

Una timeline puede mostrar, por ejemplo:

- Hora de apertura de un documento malicioso.
- Ejecución de un proceso sospechoso.
- Creación o modificación de ficheros.
- Conexiones de red.
- Generación de alertas.
- Activación de reglas BIOC o correlation rules.
- Eventos informativos relacionados con el ataque.

Ejemplo simplificado:

10:01 - El usuario abre un documento sospechoso.  
10:02 - winword.exe lanza powershell.exe.  
10:03 - powershell.exe descarga un ejecutable.  
10:04 - Se ejecuta payload.exe.  
10:05 - Se crea una clave de persistencia.  
10:06 - Se establece conexión con una IP externa.  
10:07 - Cortex XDR genera una alerta.

La Timeline es especialmente útil para determinar si una acción ocurrió antes o después de otra, identificar la progresión del ataque y reconstruir el contexto temporal de la actividad maliciosa.

En el contexto de Forensics también puede existir una *Investigation Timeline*, que muestra artefactos forenses etiquetados dentro de una investigación. En ambos casos, la idea fundamental es la misma: organizar la evidencia en orden temporal.

#### 2.1.5. Diferencia entre Causality Chain y Timeline

Una comparación importante para el examen es la diferencia entre *Causality Chain* y *Timeline*.

| Concepto               | Función principal                                                                                                        |
|------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>Causality Chain</i> | Muestra relaciones causa-efecto entre procesos, eventos y alertas. Permite entender qué originó la actividad sospechosa. |
| <i>Timeline</i>        | Muestra los eventos en orden cronológico. Permite entender cuándo ocurrió cada acción.                                   |

Por ejemplo, si se quiere saber si powershell.exe fue lanzado por winword.exe, se debe revisar la *Causality Chain*. Si se quiere saber si una conexión externa ocurrió antes o después de la ejecución de un payload, se debe revisar la *Timeline*.

#### 2.1.6. Trampas típicas

- Pensar que investigar una alerta consiste solo en mirar el nombre y la severidad.
- Confundir *Forensics* con *Causality Chain*.
- Pensar que ITDR se centra en malware, cuando realmente se centra en identidad.
- Confundir *Causality Chain* con *Timeline*.
- Pensar que la *Timeline* solo muestra alertas; también puede mostrar eventos, BIOC's informativos, correlation rules y otros elementos relacionados.
- Analizar solo el nodo de alerta y no revisar procesos padre, procesos hijos y artefactos relacionados.

## 2.2. Identify and analyze security events and incidents

El apartado **Identify and analyze security events and incidents** evalúa la capacidad del analista para identificar y analizar eventos de seguridad, alertas e incidentes dentro de Cortex XDR.

El objetivo principal no es únicamente observar que existe una alerta o un incidente, sino comprender qué ha ocurrido, qué activos están afectados, qué usuario o entidad está implicada, cuál puede ser la causa raíz y qué impacto potencial tiene la actividad detectada.

### 2.2.1. Diferencia entre evento, alerta e incidente

Para analizar correctamente la información en Cortex XDR es necesario distinguir tres conceptos básicos:

- **Evento:** actividad registrada por una fuente de datos. Puede ser una ejecución de proceso, una conexión de red, un inicio de sesión, una modificación de fichero o cualquier otra acción observada. Un evento no tiene por qué ser malicioso.
- **Alerta:** notificación generada cuando Cortex XDR o una fuente integrada detecta una actividad sospechosa, maliciosa o relevante desde el punto de vista de seguridad.
- **Incidente (caso):** agrupación de alertas, activos y artefactos relacionados que representan una posible actividad maliciosa o una investigación común.

La **idea clave** es que un evento es información bruta o actividad observada; una alerta es una detección relevante; y un incidente es una unidad de investigación que agrupa contexto relacionado.

### 2.2.2. Objetivo del análisis

El análisis de eventos e incidentes busca responder preguntas como:

- ¿Qué actividad se ha detectado?
- ¿Es una actividad normal, sospechosa o maliciosa?
- ¿Qué endpoint, usuario o activo está afectado?
- ¿Qué proceso, fichero, hash, IP o dominio está implicado?
- ¿Existen otras alertas relacionadas?
- ¿La actividad forma parte de una cadena de ataque?
- ¿Cuál es el alcance del incidente?
- ¿Cuál es el impacto potencial?

- ¿Debe escalar, cerrarse o responderse mediante acciones de contención y remediación?

Por tanto, analizar un incidente no consiste únicamente en revisar el nombre de la alerta o su severidad. El analista debe revisar el contexto completo y tomar una decisión basada en evidencia.

### 2.2.3. Elementos que deben revisarse en un incidente

Al investigar un incidente en Cortex XDR, es importante revisar los siguientes elementos:

- **Nombre del incidente:** proporciona una primera indicación de la actividad detectada.
- **Severidad:** indica la gravedad asignada al incidente o a sus alertas.
- **Score:** ayuda a priorizar el incidente frente a otros.
- **Estado:** permite conocer si el incidente está nuevo, en investigación, resuelto o en otro estado operativo.
- **Alertas asociadas:** muestran las detecciones que forman parte del incidente.
- **Fuentes de alerta:** indican si las alertas proceden del agente, Analytics, BIOC, IOC, firewall, reglas de correlación o fuentes externas.
- **Hosts afectados:** permiten identificar los endpoints o sistemas implicados.
- **Usuarios afectados:** ayudan a determinar si hay compromiso de identidad o abuso de credenciales.
- **Key assets and artifacts:** incluyen hosts, usuarios, hashes, procesos, ficheros, IPs, dominios y otros elementos relevantes.
- **MITRE ATT&CK:** permite clasificar la actividad según tácticas y técnicas conocidas.
- **Causality Chain:** muestra relaciones causa-efecto entre procesos y eventos.
- **Timeline:** muestra la secuencia temporal de la actividad.

### 2.2.4. Scope e impact

Dos conceptos importantes en el análisis de incidentes son *scope* e *impact*.

- **Scope:** indica el alcance del incidente. Responde a preguntas como cuántos hosts, usuarios, procesos, ficheros o conexiones están afectados.

- **Impact:** indica el posible daño o riesgo asociado al incidente. Puede incluir ejecución de malware, persistencia, movimiento lateral, compromiso de credenciales, exfiltración de datos o afectación de sistemas críticos.

Por ejemplo, una alerta en un único equipo de pruebas puede tener un alcance limitado. Sin embargo, la misma alerta en un controlador de dominio, un servidor de producción o una cuenta privilegiada puede tener un impacto mucho mayor.

#### 2.2.5. Análisis de assets y artifacts

Los *assets* y *artifacts* son elementos fundamentales para analizar un incidente.

- **Assets:** activos afectados, como endpoints, servidores, usuarios o cuentas.
- **Artifacts:** elementos técnicos asociados al incidente, como hashes, ficheros, rutas, procesos, IPs, dominios o URLs.

El análisis de estos elementos permite pivotar durante la investigación. Por ejemplo, si un incidente contiene un hash sospechoso, el analista puede buscar si ese hash aparece en otros endpoints. Si aparece una IP externa sospechosa, puede investigarse si otros hosts se han comunicado con esa IP. Si un usuario está implicado, puede revisarse si presenta actividad anómala o si ha accedido a otros sistemas sensibles.

#### 2.2.6. Estados de un Case (igual para issues)

Un *Case* también tiene estados propios dentro del flujo de investigación. Como un case agrupa varias issues relacionadas, su estado refleja el estado general de la investigación.

Los estados principales de un *Case* son:

**New** El case se ha creado recientemente, ya sea automáticamente a partir de una issue o manualmente por un usuario. Todavía no ha sido revisado en profundidad.

““

**Under Investigation** El case está siendo investigado activamente. El analista está revisando las issues asociadas, los activos afectados, los artefactos, la timeline, la causality chain y el posible impacto.

**Reopened** El case estaba cerrado, pero se ha reabierto por la aparición de nueva actividad relacionada o por una nueva issue que coincide con el caso.

**Resolved** El case ha sido cerrado tras finalizar la investigación. Al resolverlo, debe indicarse un motivo de resolución. ““

### 2.2.7. Motivos de resolución de un Case

Los motivos de resolución de un *Case* son equivalentes a los de una *Issue*:

**Resolved – True Positive** El case representa una amenaza real confirmada y gestionada.

““

**Resolved – False Positive** El case no representa una amenaza real.

**Resolved – Security Testing** El case está relacionado con pruebas de seguridad autorizadas.

**Resolved – Known Issue** El case corresponde a un problema ya conocido o previamente gestionado.

**Resolved – Duplicate Case** El case duplica otro caso existente.

**Resolved – Risk Accepted** El riesgo asociado al case es aceptado por la organización.

**Resolved – Auto Resolved** El case ha sido cerrado automáticamente por Cortex XDR, normalmente porque todas las issues asociadas han sido resueltas. ““

### 2.2.8. Estados personalizados

Además de los estados principales, Cortex XDR puede permitir estados personalizados definidos por la organización. Estos estados dependen de la configuración del entorno.

Algunos ejemplos de estados personalizados podrían ser:

- *Triage*
- *Pending Review*
- *Escalated*
- *Waiting for Customer*
- *Pending Remediation*

Estos estados no son universales, sino que dependen de cómo la organización haya configurado su flujo de trabajo.

### 2.2.9. Resumen

El apartado **Identify and analyze security events and incidents** evalúa la capacidad del analista para identificar eventos, alertas e incidentes, analizar su contexto y determinar si representan una amenaza real.

La frase clave para memorizar es:

Analizar un incidente en Cortex XDR consiste en revisar alertas, assets, artifacts, usuarios, hosts, procesos, timeline, causality y contexto MITRE para determinar la causa raíz, el alcance, el impacto y la clasificación correcta del incidente.

## 2.3. Respond to incidents

El apartado **Respond to incidents** evalúa la capacidad del analista para responder ante una actividad sospechosa o maliciosa detectada en Cortex XDR.

La idea principal de este apartado es:

Investigar responde a la pregunta **qué ha ocurrido**; responder responde a la pregunta **qué hago para contener, erradicar, recuperar y cerrar correctamente la investigación**.

### 2.3.1. Objetivos de la respuesta

La respuesta ante un incidente o case debe perseguir varios objetivos:

- **Contención:** limitar el impacto de la amenaza y evitar propagación.
- **Erradicación:** eliminar ficheros, procesos, persistencia o cambios maliciosos.
- **Recuperación:** devolver el sistema a un estado seguro y operativo.
- **Validación:** comprobar que la actividad maliciosa ha sido eliminada.
- **Cierre:** resolver la issue o el case con el motivo adecuado.
- **Mejora:** ajustar políticas, reglas, automatizaciones o controles para evitar recurrencia.

### 2.3.2. Available response actions

Las **available response actions** son acciones que Cortex XDR permite ejecutar durante una investigación para contener, investigar o remediar una amenaza.

Las acciones más importantes son:

**Isolate endpoint** Aísla un endpoint de la red para impedir que se comuniquen con otros sistemas internos o externos.

- Unisolate endpoint** Restaura la conectividad del endpoint una vez que se ha validado que ya no representa un riesgo o que la contención ya no es necesaria.
- Live Terminal** Permite iniciar una sesión remota sobre un endpoint para investigar y responder. Puede utilizarse para revisar ficheros, procesos activos y ejecutar comandos del sistema operativo o comandos Python.
- Run script** Permite ejecutar scripts en endpoints gestionados para recoger información, automatizar tareas de investigación o aplicar acciones de remediación.
- Retrieve endpoint files** Permite recuperar ficheros desde uno o varios endpoints para analizarlos en Cortex XDR o en otras herramientas.
- Retrieve support files** Recoge logs y datos diagnósticos del endpoint para análisis técnico o troubleshooting.
- Terminate process** Detiene un proceso activo en un endpoint. Es útil cuando un proceso malicioso o sospechoso sigue ejecutándose.
- Quarantine file** Aísla un fichero para impedir su ejecución, pero lo conserva para análisis posterior.
- Destroy file** Elimina permanentemente un fichero de un endpoint. Es una acción destructiva e irreversible, por lo que debe aplicarse cuando existe suficiente evidencia de que el fichero es malicioso.
- Search and Destroy** Busca un fichero por hash o ruta en endpoints y permite eliminar sus instancias. Se utiliza cuando se conoce un fichero malicioso y se quiere actuar a escala.
- Block list** Permite bloquear la ejecución de un fichero, normalmente mediante hash, para impedir que se ejecute en endpoints protegidos.
- Allow list** Permite permitir explícitamente la ejecución de un fichero que podría estar siendo bloqueado o detectado incorrectamente.
- Malware scan** Ejecuta un escaneo en el endpoint para buscar malware.
- Resolve issue or case** Cierra la issue o el case con un motivo de resolución adecuado, como *True Positive*, *False Positive*, *Security Testing*, *Known Issue*, *Duplicate Case* o *Risk Accepted*. ““

### 2.3.3. Action Center

El **Action Center** es la ubicación central desde la que se puede hacer seguimiento de las acciones ejecutadas sobre endpoints. Permite revisar acciones de investigación, respuesta y mantenimiento.

Desde el *Action Center* se pueden revisar, entre otras cosas:

- Acciones pendientes, en curso o completadas.
- Endpoints aislados.
- Ficheros en cuarentena.
- Acciones de búsqueda y destrucción de ficheros.
- Block lists y allow lists.
- Recuperación de ficheros.
- Acciones ejecutadas por scripts o tareas.

#### 2.3.4. Containment, eradication and recovery

Las acciones de respuesta pueden clasificarse según su finalidad:

**Containment** Busca limitar el daño y evitar propagación. Ejemplos: aislar endpoint, terminar proceso, bloquear hash o impedir ejecución.

**Eradication** Busca eliminar la amenaza del entorno. Ejemplos: destruir fichero, eliminar persistencia, revertir cambios maliciosos o remediar claves de registro.

**Recovery** Busca devolver el activo a un estado seguro y operativo. Ejemplos: desaislar endpoint, restaurar ficheros, validar limpieza y confirmar que no quedan indicadores activos.

#### 2.3.5. Remediation suggestions

Las **remediation suggestions** son recomendaciones generadas por Cortex XDR para revertir cambios maliciosos asociados a una cadena de causalidad.

Cuando Cortex XDR detecta una *causality chain* maliciosa, puede analizar los cambios realizados en el endpoint y sugerir acciones de remediación. Estas sugerencias ayudan al analista a evitar una búsqueda manual de todos los ficheros, procesos o claves de registro afectados.

Las sugerencias de remediación pueden incluir:

- **Delete File:** eliminar un fichero creado por actividad maliciosa.
- **Restore File:** restaurar un fichero modificado o afectado.
- **Rename File:** renombrar un fichero implicado.
- **Delete Registry Value:** eliminar un valor de registro creado o modificado por la actividad maliciosa.
- **Restore Registry Value:** restaurar un valor de registro.
- **Terminate Process:** finalizar un proceso concreto.

- **Terminate Causality:** finalizar la cadena de procesos relacionada con la actividad maliciosa.

La idea importante es que las *remediation suggestions* no son recomendaciones genéricas. Están relacionadas con la evidencia observada en la causality chain y con los cambios detectados en procesos, ficheros o registro.

### 2.3.6. Diferencia entre response action y remediation suggestion

Es importante no confundir estos dos conceptos:

- **Response action:** acción que el analista puede ejecutar, como aislar un endpoint, recuperar un fichero, terminar un proceso o destruir un fichero.
- **Remediation suggestion:** recomendación generada por Cortex XDR sobre qué acciones aplicar para revertir cambios maliciosos.

### 2.3.7. Automated responses

Las *automated responses* permiten automatizar parte del proceso de investigación y respuesta mediante reglas, playbooks, scripts, comandos, integraciones o acciones rápidas.

En el modelo actual de Cortex XDR, la automatización puede ejecutarse de forma manual durante una investigación o de forma automática cuando se cumplen determinadas condiciones.

Los principales mecanismos de automatización son:

**Playbooks** Flujos de trabajo compuestos por tareas, scripts, comandos, integraciones, condiciones y sub-playbooks. Permiten estandarizar y automatizar procesos de investigación y respuesta.

**Automation rules** Reglas que ejecutan acciones automáticamente cuando se cumplen condiciones predefinidas. Su lógica puede entenderse como *WHEN / IF / THEN*: cuando ocurre un evento, si se cumple una condición, entonces se ejecuta una acción.

**Quick Actions** Acciones puntuales y rápidas que permiten responder sin construir un playbook complejo.

**Scripts and commands** Pueden ejecutarse manualmente o formar parte de playbooks para enriquecer información, remediar o interactuar con sistemas externos.

**Integrations** Permiten conectar Cortex XDR con herramientas externas, como sistemas de ticketing, SIEM, EDR, firewalls, correo, identidad o plataformas cloud.

### 2.3.8. Playbooks

Un *playbook* es un flujo automatizado o semiautomatizado de tareas. Puede utilizarse para:

- Enriquecer una issue con información adicional.
- Consultar reputación de hashes, IPs o dominios.
- Crear o actualizar tickets.
- Ejecutar scripts.
- Solicitar aprobación manual.
- Ejecutar acciones de contención o remediación.
- Actualizar campos de una issue o case.
- Documentar pasos de investigación.

Los playbooks pueden incluir tareas automáticas y tareas manuales. Esto permite mantener control humano en acciones de alto impacto, como aislar un servidor crítico o destruir un fichero.

### 2.3.9. Automation rules

Las **automation rules** permiten lanzar automatizaciones cuando se cumplen condiciones.

Un ejemplo de regla sería:

```
WHEN an issue is created
IF severity is Critical
THEN assign the case to Tier 2 and run a containment playbook
```

### 2.3.10. Resolution tab and Resolution Center

La pestaña *Resolution* muestra acciones de remediación recomendadas, playbooks recomendados y acciones pendientes, en progreso o completadas.

Aunque muchas acciones son específicas de una issue, pueden sincronizarse con el *Resolution Center* del case vinculado. Esto permite gestionar la remediación desde una visión más amplia del caso completo.

Para el examen, la idea clave es:

La pestaña Resolution y el Resolution Center ayudan a centralizar las acciones de remediación asociadas a issues y cases.

### 2.3.11. Resumen

El apartado *Respond to incidents* evalúa la capacidad del analista para ejecutar acciones de contención, erradicación, recuperación y cierre en Cortex XDR.

La frase clave para memorizar es:

Responder a un incidente en Cortex XDR consiste en ejecutar acciones sobre endpoints, procesos, ficheros, issues o casos para contener la amenaza, remediar los cambios maliciosos, automatizar tareas repetitivas y cerrar la investigación con el motivo adecuado.

## 2.4. Identify and explain exclusions and exceptions

El apartado *Identify and explain exclusions and exceptions* evalúa la capacidad del analista para comprender, diferenciar y aplicar correctamente exclusiones y excepciones dentro de Cortex XDR.

La idea principal de este apartado es:

Las exclusions y exceptions sirven para reducir ruido, ajustar detecciones o modificar el comportamiento de protección, pero no son lo mismo.

### 2.4.1. Diferencia general entre exclusion y exception

La diferencia entre *exclusion* y *exception* es uno de los puntos más importantes para el examen.

**Exclusion** Se utiliza para suprimir issues, alertas o detecciones que coinciden con unos criterios determinados. Su objetivo principal es reducir ruido y evitar que determinadas detecciones aparezcan en casos, búsquedas o vistas de investigación.

**Exception** Se utiliza para modificar el comportamiento de una regla, política, módulo de protección, veredicto o flujo de remediación. Su objetivo no es simplemente ocultar una detección, sino también quitar la protección.

### 2.4.2. Issue Exclusions

Una *Issue Exclusion* es una regla que permite suprimir issues futuras que coincidan con determinados criterios.

Cuando se configura una *Issue Exclusion*, Cortex XDR deja de mostrar y almacenar futuras issues que cumplan esos criterios. Además, esas issues no se incorporan a casos ni aparecen en resultados de búsqueda.

Esto resulta útil cuando una detección ya ha sido investigada y se ha confirmado que corresponde a una actividad benigna, esperada o irrelevante para la organización.

### 2.4.3. Issue Exceptions

Una *Issue Exception* representa una aceptación temporal, documentada y controlada de una issue real cuya remediación no puede realizarse inmediatamente. Se utiliza cuando la organización reconoce que existe un riesgo o problema, pero decide aplazar su corrección por motivos técnicos, operativos o de negocio.

### 2.4.4. Endpoint policy exceptions o Agent exceptions

Las *Endpoint policy exceptions* o *Agent exceptions* permiten modificar cómo se aplica la protección del agente de Cortex XDR sobre endpoints.

A diferencia de una *Issue Exclusion*, que suprime issues, una excepción de endpoint puede modificar el comportamiento de prevención, inspección o evaluación.

Ejemplos de uso:

- Excluir una ruta de evaluación.
- Permitir un proceso concreto.
- Permitir un fichero específico.
- Permitir un firmante digital conocido.
- Desactivar una regla concreta de Behavioral Threat Protection.
- Aplicar una excepción recomendada por soporte.

### 2.4.5. IOC/BIOC rule exceptions

Las *IOC/BIOC rule exceptions* permiten mantener activa una regla IOC o BIOC, pero excluir indicadores concretos que generan falsos positivos.

Esto es útil cuando una regla es válida en general, pero un caso particular debe ser ignorado.

Ejemplo:

- Existe un BIOC que detecta ejecución sospechosa de PowerShell.
- Una herramienta administrativa interna usa PowerShell de forma legítima.
- No se quiere desactivar toda la regla.
- Se crea una excepción para ese proceso, ruta, usuario o argumento concreto.

La idea clave es:

**IOC/BIOC exception = la regla sigue activa, pero ignora un indicador concreto.**

Esto es preferible a desactivar completamente una regla, ya que permite mantener cobertura de detección sobre el resto de comportamientos o indicadores.

#### 2.4.6. Disable Prevention Rules

Una *Disable Prevention Rule* permite evitar que Cortex XDR aplique acciones de prevención sobre determinados elementos.

Puede utilizarse para excluir de prevención elementos como:

- Hashes.
- Rutas de ficheros o carpetas.
- Firmantes digitales.
- Certificados.
- Líneas de comando.
- Procesos concretos.

La finalidad es evitar que un proceso o fichero legítimo sea bloqueado, terminado o prevenido por un módulo de protección.

Sin embargo, una *Disable Prevention Rule* no debe confundirse con una *Issue Exclusion*. La primera afecta a la prevención; la segunda afecta al reporting de issues.

**Disable Prevention Rule = evita acciones de prevención sobre elementos concretos.**

**Issue Exclusion = suprime issues en Cortex XDR.**

Una regla de este tipo debe usarse con cuidado, porque puede reducir la protección frente a amenazas reales si se define de forma demasiado amplia.

#### 2.4.7. Support Exception Rules

Las *Support Exception Rules* son excepciones proporcionadas por el soporte de Palo Alto Networks.

Normalmente se aplican cuando soporte identifica que una excepción concreta es necesaria para resolver un falso positivo, una incompatibilidad o un comportamiento no deseado del agente.

#### 2.4.8. Cuándo usar cada opción

**Issue Exclusion** Se usa cuando una issue benigna o aceptada genera ruido y no se quiere que aparezca más en casos o búsquedas.

**Issue Exception** Se usa cuando una issue real no puede remediarse inmediatamente y se acepta temporalmente con justificación.

**Hash Exception** Se usa para sobrescribir el veredicto de un fichero concreto por hash.

**IOC/BIOC Rule Exception** Se usa cuando una regla IOC o BIOC debe seguir activa, pero se quiere excluir un indicador concreto.

**Disable Prevention Rule** Se usa cuando se quiere evitar una acción de prevención sobre un proceso, hash, ruta o firmante concreto.

**Disable Injection and Prevention Rule** Se usa para aplicar un bypass temporal de prevención e inyección sobre un proceso concreto.

**Support Exception Rule** Se usa cuando soporte proporciona una excepción concreta.

**Pause Endpoint Protection** Se usa cuando se necesita desactivar temporalmente capacidades de protección del agente. ““

#### 2.4.9. Resumen

El apartado *Identify and explain exclusions and exceptions* evalúa si el analista entiende cómo reducir ruido y ajustar el comportamiento de Cortex XDR sin perder visibilidad ni protección de forma innecesaria.

La idea principal es:

Las exclusions suprimen issues o detecciones para reducir ruido, mientras que las exceptions modifican cómo Cortex XDR evalúa, previene, permite, bloquea o remedia determinados elementos.

La frase clave para memorizar es:

En Cortex XDR, una exclusion sirve principalmente para ocultar o suprimir una issue, mientras que una exception sirve para modificar el comportamiento de una regla, política, módulo de protección, veredicto o remediación. Confundir ambos conceptos puede provocar pérdida de visibilidad o debilitamiento de la protección.

## 3. Data Analysis 28 %

### 3.1. Use XQL to query datasets

XQL significa **Cortex Query Language**. Es el lenguaje de consulta utilizado en Cortex XDR para buscar información en los datos almacenados en la plataforma. Permite analizar eventos de endpoint, actividad de red, logs de fuentes externas, artefactos, procesos, ficheros, usuarios e indicadores de compromiso.

La idea general de una consulta XQL es:

- Seleccionar un dataset
- Filtrar datos
- Elegir campos relevantes

- Ordenar o agregar resultados
- Analizar la información obtenida.

### 3.1.1. Concepto de dataset

Un **dataset** es una fuente de datos consultable mediante XQL. Cada dataset tiene un nombre único y una serie de campos asociados.

El dataset más importante es **xdr\_data**, el cual contiene eventos reportados por el agente de Cortex XDR y logs procedentes de distintas fuentes. Es uno de los datasets principales para investigar actividad de endpoints, procesos, ficheros y red.

Además de datasets, Cortex XDR también puede ofrecer **presets**. Un preset es una vista o esquema preparado para facilitar la investigación de determinados tipos de actividad. Mientras que un dataset suele representar una fuente de datos amplia, un preset está orientado a facilitar consultas sobre una categoría concreta de eventos.

**Dataset = fuente de datos consultable.**

**Preset = vista preparada o esquema común para investigar un tipo de actividad.**

### 3.1.2. Estructura básica de una consulta XQL

Una consulta XQL suele comenzar identificando la fuente de datos. Después se añaden fases o *stages*, separadas mediante el operador pipe |.

La estructura general es:

```
dataset = nombre_dataset
| stage_1
| stage_2
| stage_3
```

Ejemplo básico:

```
dataset = xdr_data
| filter agent_hostname = "HOST-01"
| fields _time, event_type, event_sub_type, agent_hostname
| sort desc _time
| limit 100
```

Esta consulta busca eventos en **xdr\_data**, filtra por un endpoint concreto, selecciona algunos campos, ordena los resultados por tiempo descendente y limita la salida a 100 registros.

### 3.1.3. Stages principales

Los **stages** son las fases de una consulta XQL. Cada stage realiza una operación concreta sobre los datos.

El stage **dataset** indica la fuente de datos que se quiere consultar.

```
dataset = xdr_data
```

Es normalmente el primer elemento de una consulta, salvo que se utilice un stage **config**, que debe aparecer al principio.

El stage **filter** permite filtrar registros. Solo se devuelven aquellos eventos que cumplen la condición indicada.

```
dataset = xdr_data
| filter agent_hostname = "HOST-01"
```

Ejemplo de búsqueda de PowerShell:

```
dataset = xdr_data
| filter actor_process_image_path contains "powershell.exe"
```

El stage **fields** permite seleccionar qué campos o columnas se devuelven en los resultados. También permite renombrar campos mediante **as**.

```
dataset = xdr_data
| fields _time, agent_hostname, actor_process_image_path as process_path
```

Una idea importante es que, si se utiliza **fields**, los stages posteriores solo pueden operar sobre los campos seleccionados. Además, si un campo se renombra con **as**, los stages posteriores deben usar el alias.

```
dataset = xdr_data
| fields _time, agent_hostname, actor_process_image_path as process_path
| filter process_path contains "powershell.exe"
```

El stage **sort** ordena los resultados. Puede utilizarse en orden ascendente o descendente.

```
dataset = xdr_data
| fields _time, agent_hostname, event_type
| sort desc _time
| limit 100
```

En investigaciones de seguridad suele utilizarse **sort desc \_time** para ver primero los eventos más recientes.

El stage **limit** limita el número máximo de registros devueltos.

```
dataset = xdr_data
| limit 100
```

El stage **comp** se utiliza para agrupar datos y realizar cálculos estadísticos sobre grupos de registros.

Ejemplo: contar procesos únicos por host.

```
dataset = xdr_data
| comp count_distinct(actor_process_image_path) as unique_processes by agent_hostname
| sort desc unique_processes
| limit 20
```

El stage **dedup** elimina registros duplicados en función de uno o varios campos. Se utiliza para reducir ruido y obtener valores únicos, como hashes únicos, procesos únicos o hosts únicos.

```
dataset = xdr_data
| fields _time, agent_hostname, action_file_sha256
| dedup action_file_sha256 by desc _time
```

El stage **alter** permite crear un campo nuevo o modificar el valor de un campo existente.

```
dataset = xdr_data
| alter suspicious = if(actor_process_command_line contains "EncodedCommand", "yes", "no")
| filter suspicious = "yes"
```

El stage **join** combina resultados de dos consultas utilizando una condición de relación. Es conceptualmente similar a un **JOIN** de SQL.

El stage **union** combina dos conjuntos de resultados en uno solo.

El stage **bin** permite agrupar eventos por intervalos, normalmente de tiempo. Se utiliza especialmente para series temporales y gráficos.

```
dataset = xdr_data
| bin _time span = 1h
| comp count() as events by _time
```

El stage **config** permite configurar el comportamiento de la consulta. Si se utiliza, debe aparecer al principio de la query. Un uso habitual es definir el marco temporal mediante **timeframe**.

Para el examen, es importante recordar que utilizar un marco temporal adecuado mejora el rendimiento y reduce el ruido.

**Las consultas deben limitar el periodo temporal siempre que sea posible.**

#### 3.1.4. Campos habituales en investigaciones XQL

No es necesario memorizar todos los campos de **xdr\_data**, pero sí reconocer campos habituales de investigación.

Algunos campos comunes son:

`_time`: tiempo del evento.  
`_insert_time`: tiempo de inserción del evento en la plataforma.  
`event_type`: tipo de evento.  
`event_sub_type`: subtipo de evento.  
`agent_hostname`: nombre del endpoint.  
`agent_id`: identificador del agente.  
`actor_process_image_path`: ruta del proceso actor.  
`actor_process_command_line`: línea de comandos del proceso actor.  
`actor_process_image_sha256`: hash SHA256 del proceso actor.  
`action_file_path`: ruta de fichero afectado.  
`action_file_sha256`: hash SHA256 de fichero afectado.  
`action_remote_ip`: IP remota.  
`action_remote_port`: puerto remoto.  
`action_local_ip`: IP local.  
`action_total_upload`: volumen subido.  
`action_total_download`: volumen descargado.

### 3.1.5. Consultas típicas

**Buscar eventos recientes de un host** Esta consulta sirve para revisar la actividad reciente de un endpoint concreto.

```
dataset = xdr_data
| filter agent_hostname = "HOST-01"
| fields _time, event_type, event_sub_type, agent_hostname
| sort desc _time
| limit 100
```

**Buscar ejecución de PowerShell** Esta consulta sirve para localizar ejecuciones de PowerShell en endpoints.

```
dataset = xdr_data
| filter actor_process_image_path contains "powershell.exe"
| fields _time, agent_hostname, actor_process_image_path, actor_process_command_line
| sort desc _time
| limit 100
```

**Buscar PowerShell con comando sospechoso** Esta consulta sirve para buscar posibles ejecuciones ofuscadas o codificadas de PowerShell.

```
dataset = xdr_data
| filter actor_process_image_path contains "powershell.exe"
| filter actor_process_command_line contains "EncodedCommand"
| fields _time, agent_hostname, actor_process_command_line
| sort desc _time
| limit 100
```

**Buscar un hash** Esta consulta sirve para saber si un fichero concreto ha aparecido en otros endpoints.

```
dataset = xdr_data
| filter action_file_sha256 = "HASH_SHA256_AQUI"
| fields _time, agent_hostname, action_file_path, action_file_sha256
| sort desc _time
| limit 100
```

**Buscar conexiones a una IP** Esta consulta sirve para investigar comunicaciones hacia una IP concreta.

```
dataset = xdr_data
| filter action_remote_ip = "8.8.8.8"
| fields _time, agent_hostname, actor_process_image_path, action_remote_ip, action_remote_po
| sort desc _time
| limit 100
```

**Contar eventos por host** Esta consulta permite identificar hosts con mayor volumen de eventos.

```
dataset = xdr_data
| comp count() as total_events by agent_hostname
| sort desc total_events
| limit 20
```

**Contar conexiones por IP remota** Esta consulta permite identificar las IPs remotas más frecuentes.

```
dataset = xdr_data
| filter action_remote_ip != null
| comp count() as connections by action_remote_ip
| sort desc connections
| limit 20
```

### 3.1.6. Resultados de una consulta XQL

Al ejecutar una consulta, puedes revisar resultados en formato tabla, gráfico o vista avanzada. La documentación indica que los resultados pueden filtrarse por columnas, visualizarse como tabla, gráfico o formato avanzado, exportarse a TSV y mostrarse como **RAW**, **JSON** o **TREE**. También permite guardar consultas como BIOC Rule, Correlation Rule, Query to Library o Widget to Library cuando sea compatible.

### 3.1.7. Trampas típicas

- Pensar que XQL solo sirve para endpoints.
- Confundir datasets con presets.
- Usar **fields** demasiado pronto y luego intentar filtrar por un campo que ya no está disponible.
- Renombrar un campo con **as** y después seguir usando el nombre original.
- Pensar que **limit** filtra por una condición de seguridad.
- Confundir **join** con **union**.
- Ejecutar consultas amplias sin timeframe ni filtros.
- Ordenar o agregar grandes volúmenes de datos sin limitar primero el alcance.
- No revisar los resultados para pivotar hacia causality, timeline o reglas.

## 3.2. Identify and explain components of XQL data structure

La idea principal es:

XQL trabaja sobre fuentes de datos estructuradas. Una consulta selecciona una fuente de datos, aplica stages separados por pipes, utiliza campos definidos en un schema y devuelve resultados útiles para investigación, hunting, dashboards o reglas.

### 3.2.1. Syntax and schema

**Operadores importantes** XQL utiliza operadores para comparar, filtrar y buscar valores.

Algunos operadores relevantes son:

- =
- !=
- >
- <
- >=
- <=
- and

- or
- not
- in
- not in
- contains
- not contains
- ~=
- incidr
- not incidr

Ejemplo con contains:

```
dataset = xdr_data
| filter actor_process_image_path contains "powershell.exe"
```

Ejemplo con in:

```
dataset = xdr_data
| filter action_remote_port in (80, 443, 8080)
```

Ejemplo con expresión regular:

```
dataset = xdr_data
| filter action_process_image_name ~= ".*\.exe"
```

Ejemplo con rango CIDR:

```
dataset = xdr_data
| filter action_remote_ip incidr "192.168.1.0/24"
```

El **schema** es la estructura de campos de un dataset. Define qué campos existen, qué tipo de datos contienen y qué representa cada campo. Si no se conoce el schema, no se sabe qué campos pueden utilizarse en **filter**, **fields**, **comp** o cualquier otro stage.

Ejemplos de campos habituales en **xdr\_data**:

```
_time
_product
_vendor
_raw_json
agent_hostname
agent_id
event_type
action_file_sha256
```

**Tipos de datos** Los campos de un schema pueden tener distintos tipos de datos.

Tipos frecuentes:

- STRING
- INTEGER
- BOOLEAN
- RECORD
- REPEATED
- NULLABLE

Ejemplos conceptuales:

- agent\_hostname: string.
- agent\_os\_type: integer.
- agent\_is\_vdi: boolean.
- \_raw\_json: record.

**Campos de sistema** Los campos que empiezan por guion bajo suelen ser campos de sistema.

Campos importantes:

- \_time: timestamp del evento.
- \_insert\_time: momento en que el dato se insertó en Cortex XDR.
- \_product: producto de origen del dato.
- \_vendor: vendor de origen del dato.
- \_raw\_json: dato bruto en formato JSON.
- \_raw\_log: dato bruto en formato texto.

**event\_type y event\_sub\_type** El campo `event_type` identifica el tipo general de evento. El campo `event_sub_type` proporciona una clasificación más concreta.

Tipos de eventos relevantes:

- Process
- Network
- File

- Registry
- Injection
- LoadImage
- UserStatusChange
- Causality
- HostStatusChange
- AgentStatusChange
- WindowsEventLog
- Device
- HostFirewall

### 3.2.2. Data Sources

**Concepto de data source** Una **data source** es la fuente contra la que se ejecuta una consulta XQL. Puede ser un dataset, un preset, un lookup dataset u otro conjunto de datos disponible según la licencia y configuración del entorno.

**Built-in datasets** Los **built-in** datasets son datasets integrados de Cortex XDR, como **xdr\_data**.

**Custom datasets** Los custom datasets son datasets creados a partir de datos ingeridos o generados por la organización.

Pueden proceder de:

- Fuentes externas.
- XDR Collectors.
- Integraciones.
- Bases de datos.
- Logs personalizados.
- Procesos internos de ingesta.

Ejemplo conceptual:

```
dataset = mi_dataset_custom
| filter usuario = "rafa"
```

Un **lookup dataset** funciona como una tabla auxiliar de referencia. Sirve para comparar o enriquecer los resultados de una consulta.

Ejemplos:

- Lista de IPs críticas.
- Lista de dominios permitidos.
- Lista de usuarios VIP.
- Lista de hashes maliciosos.
- Lista de subredes internas.

**Fuentes externas y XDR Collectors** Cortex XDR puede ingerir datos desde distintas fuentes de Palo Alto Networks y de terceros. Una vez ingeridos, esos datos pueden quedar disponibles para consultas XQL en datasets o presets.

Fuentes habituales:

- Cortex XDR Agent.
- Datos EDR.
- Palo Alto Networks NGFW.
- Panorama.
- Strata Logging Service.
- Prisma Access.
- Cloud logs.
- Identity providers.
- SaaS logs.
- Third-party logs.
- Database collectors.
- XDR Collectors.

Para el examen:

**XQL no se limita a consultar endpoints. Puede consultar endpoint, red, cloud, identidad y fuentes externas, dependiendo de la ingesta, licencia y configuración.**

**Permisos sobre datasets** Los resultados de una consulta XQL pueden depender de los permisos del usuario. Dos usuarios pueden ejecutar la misma consulta y obtener resultados distintos si no tienen los mismos permisos sobre datasets.

**Data ingestion vs data source** No debe confundirse data ingestion con data source.

- **Data ingestion:** proceso mediante el cual los datos entran en Cortex XDR.
- **Data source:** fuente ya disponible para consultar con XQL.
- **Dataset o preset:** forma estructurada de consultar esos datos.

Ejemplo:

- Un firewall envía logs a Cortex XDR.
- Ese envío es data ingestion.
- Los logs quedan disponibles en un dataset o preset.
- Ese dataset o preset es una data source para XQL.

### 3.3. Identify and explain data query options

Evalúa si el analista conoce las distintas opciones disponibles en Cortex XDR para crear, guardar, reutilizar y programar consultas.

La idea principal es:

XQL Search permite escribir consultas manuales; Query Builder templates permiten construir búsquedas guiadas; Query Library permite guardar y reutilizar consultas; Scheduled Queries permite ejecutar consultas de forma programada.

#### 3.3.1. Pre-defined Query Builder Template

Las **pre-defined Query Builder templates** son plantillas guiadas que permiten crear consultas sin escribir XQL manualmente.

Su objetivo es facilitar búsquedas frecuentes mediante campos y filtros pre-definidos. Son útiles para analistas que quieren consultar datos rápidamente sin construir la sintaxis XQL desde cero.

Las plantillas principales son:

**Basic** Permite buscar por IP address, host name, user name y domain.

**Free text** Permite buscar una cadena de texto libre.

**Identity** Permite buscar por user name y type.

**Endpoint** Permite buscar por host name, files y processes.

**Network IP** Permite buscar por IP address y connection status.

**Cloud** Permite buscar por cloud provider y zone.

Las plantillas *Basic*, *Identity*, *Endpoint* y *Network* utilizan por defecto el dataset `xdr_data`. La plantilla *Cloud* utiliza por defecto `cloud_audit_logs`. También es posible ejecutar plantillas sobre todos los datasets, dependiendo de la configuración y del caso de uso.

Para el examen, es importante asociar cada plantilla con su finalidad:

En el modelo actual también debe tenerse presente el concepto de *XQL query entities*. Query Builder permite construir consultas guiadas para distintos tipos de entidades.

Las entidades principales son:

**Process** Búsqueda sobre ejecución e inyección de procesos por nombre, hash, path, argumentos de línea de comandos, etc.

**File** Búsqueda sobre creación y modificación de ficheros por nombre y ruta.

**Network** Búsqueda sobre actividad de red por IP, puerto, hostname, protocolo, etc.

**Image Load** Búsqueda sobre carga de módulos en procesos.

**Registry** Búsqueda sobre creación o modificación de claves, valores, rutas o datos de registro.

**Event Log** Búsqueda sobre Windows event logs y Linux system authentication logs.

**Network Connections** Búsqueda sobre conexiones de red a partir de firewall logs y endpoint raw data.

**Authentications** Búsqueda sobre eventos de autenticación por identidad, resultado y otros atributos.

**All Actions** Búsqueda transversal sobre actividad de red, registro, ficheros y procesos.

La diferencia clave es:

**Query Builder templates/entities = búsqueda guiada por tipo de entidad o campo.**

**XQL Search = consulta libre escrita manualmente con sintaxis XQL.**

Una consulta creada con template puede revisarse y modificarse. En algunos casos, Cortex XDR permite continuar la consulta en XQL para editarla directamente como una consulta XQL.

### 3.3.2. Query Library

La *Query Library* es el repositorio donde se guardan y gestionan consultas para reutilizarlas posteriormente. Útil para consultas recurrentes.

Sirve para:

- Guardar consultas útiles.
- Reutilizar búsquedas de hunting o investigación.
- Organizar consultas mediante labels.
- Compartir consultas con otros usuarios del mismo tenant.
- Estandarizar búsquedas dentro del SOC.

Una consulta puede guardarse en la Query Library desde:

- *Query Builder / XQL Search*, usando la opción *Save as* y seleccionando *Query to Library*.
- *Query Center*, seleccionando una consulta ejecutada y guardándola en la librería.

Al guardar una consulta pueden definirse elementos como:

**Name** Nombre identificativo de la consulta.

**Description** Descripción del propósito de la consulta.

**Labels** Etiquetas para clasificar y buscar consultas.

**Share with others** Opción para compartir la consulta con otros usuarios del mismo tenant.

**Private** Opción para mantener la consulta visible solo para el usuario que la guardó.

#### Diferencia entre Query Library, Query Center y Scheduled Queries

Es importante distinguir estos conceptos:

**Query Library** Repositorio de consultas guardadas para reutilizar.

**Query Center** Historial y gestión de consultas ejecutadas o programadas.

**Scheduled Queries** Consultas configuradas para ejecutarse en una fecha, hora o frecuencia determinada.

La Query Library no ejecuta automáticamente una consulta. Solo la guarda para poder reutilizarla. Para ejecutar una consulta de forma automática o recurrente, debe configurarse como *Scheduled Query*.

**Query Center** El **Query Center** permite revisar consultas ejecutadas o programadas. Puede mostrar información como:

- Usuario que ejecutó o programó la consulta.
- Duración de la ejecución.
- Execution ID.
- Número de resultados.
- Estado de la query.
- Sintaxis de la query.
- Timestamp de creación.
- Si fue creada mediante XQL Search.
- Si los resultados fueron guardados.

Los estados de una consulta pueden ser:

**Queued** La consulta está en cola esperando un slot disponible.

**Running** La consulta está ejecutándose.

**Failed** La consulta falló.

**Partially completed** La consulta se detuvo tras superar el número máximo permitido de resultados.

**Stopped** La consulta fue detenida por un administrador.

**Completed** La consulta finalizó correctamente.

**Deleted** La consulta fue eliminada o depurada.

Para el examen, es importante recordar que una consulta puede terminar como *Partially completed* si supera el máximo de resultados permitidos.

### 3.3.3. Schedule Query

Una **Scheduled Query** es una consulta configurada para ejecutarse automáticamente en una fecha concreta o de forma recurrente. Puede utilizarse para monitorizar actividades repetitivas o ejecutar búsquedas periódicas sin intervención manual.

Casos de uso:

- Buscar diariamente nuevos endpoints que ejecuten una herramienta sospechosa.

- Ejecutar cada hora una búsqueda de conexiones hacia IPs críticas.
- Programar una revisión semanal de actividad de usuarios privilegiados.
- Buscar periódicamente un hash, dominio o patrón asociado a una campaña.

Una consulta puede programarse desde Query Builder o desde Query Center. En Query Center, puede seleccionarse una query y usar la opción *Schedule*. Después se define si se ejecutará una sola vez o de forma recurrente.

Las opciones principales son:

- Ejecutar una consulta una vez en una fecha específica.
- Ejecutar una consulta de forma recurrente por fecha y hora.

La página **Scheduled Queries** permite gestionar consultas programadas y recurrentes. Desde esta vista se pueden realizar acciones como:

- Ver consultas programadas.
- Editar parámetros o frecuencia.
- Ver ejecuciones anteriores.
- Deshabilitar una scheduled query.
- Eliminar una scheduled query.

Campos importantes de una Scheduled Query:

**Issued by** Usuario que ejecutó o programó la query.

**Next execution** Próxima ejecución si la query es recurrente.

**Query description** Parámetros de la query.

**Query ID** Identificador único de la query.

**Query name** Nombre de la query o nombre autogenerado de la parent query.

**Query syntax** Sintaxis exacta de la consulta.

**Schedule time** Frecuencia o momento en el que la query está programada.

**XQL** Indica si la query fue creada mediante XQL Search.

Si una query está programada para ejecutarse una sola vez en una fecha específica, el campo *Next execution* puede aparecer como **None**. Si la query es recurrente, muestra la próxima ejecución.

#### 3.3.4. Comparación final

**Pre-defined Query Builder Template** Sirve para crear consultas guiadas sin escribir XQL manualmente.

**Query Library** Sirve para guardar, organizar, compartir y reutilizar consultas.

**Scheduled Query** Sirve para ejecutar consultas automáticamente en una fecha, hora o frecuencia determinada.

### 3.4. Use lookup tables

La idea principal es:

Una lookup table es una tabla de referencia que permite correlacionar datos externos o internos con eventos del entorno.

Ejemplos típicos de lookup tables son:

- Lista de activos críticos.
- Lista de usuarios privilegiados.
- Lista de empleados dados de baja.
- Lista de cuentas de servicio.
- Lista de IPs permitidas.
- Lista de hashes maliciosos.
- Lista de dominios sospechosos.
- Lista de subredes internas.

#### 3.4.1. Finalidad de una lookup table

Una **lookup table** permite añadir contexto a los eventos consultados. Sin una lookup, una query puede mostrar únicamente que un usuario ha realizado una acción. Con una lookup, puede saberse si ese usuario pertenece a un grupo privilegiado, si el host afectado es crítico o si la IP origen pertenece a una sede autorizada.

#### 3.4.2. Casos de uso

Los principales casos de uso de lookup tables son:

**Threat hunting** Comparar eventos con listas de hashes, IPs, dominios o usuarios sospechosos.

**Enrichment** Añadir contexto a los eventos, como criticidad del activo, propietario, departamento o rol del usuario.

**Allow lists** Evitar ruido generado por usuarios, IPs, procesos o actividades autorizadas.

**Block lists** Detectar o prevenir actividad relacionada con valores no permitidos.

**Detection rules** Utilizar listas de referencia dentro de reglas de detección.

**Response playbooks** Tomar decisiones automatizadas según si un valor aparece o no en una lookup.

**Investigación de cases e issues** Determinar si una issue afecta a activos, usuarios o IPs sensibles.

### 3.4.3. Creación de lookup tables

Una lookup table puede crearse o alimentarse de varias formas:

**Subida manual** Importando un archivo CSV, TSV o JSON desde *Dataset Management*.

**Files and Folders Collector** Mediante subida automática por el collector correspondiente.

**Resultados de una query** Guardando los resultados de una consulta XQL como lookup mediante el stage **target**.

Ejemplo conceptual de creación mediante XQL:

```
dataset = xdr_data
| filter action_file_sha256 != null
| fields action_file_sha256 as sha256, agent_hostname
| target type=lookup suspicious_hashes
```

Para el examen:

**Una lookup puede proceder de un archivo externo, de un collector o de resultados de XQL.**

### 3.4.4. Lookup dataset vs dataset normal

Es importante distinguir entre un dataset normal y una lookup table.

**Dataset normal** Almacena eventos o datos consultables de forma amplia.

**Lookup dataset** Actúa como tabla pequeña de referencia para búsquedas, filtros, joins, reglas y enriquecimiento.

Una lookup no debe usarse como repositorio masivo de eventos, sino como tabla de apoyo para correlación y enriquecimiento.

### 3.4.5. Uso de lookups en XQL

Las lookup tables pueden usarse en XQL principalmente para filtrar, correlacionar o enriquecer resultados.

**Filtrar eventos con una lookup** Ejemplo conceptual: Este patrón permite buscar eventos cuya IP remota aparece en una lista de IPs sospechosas.

```
dataset = xdr_data
| filter action_remote_ip in (dataset = suspicious_ips | fields ip)
| fields _time, agent_hostname, action_remote_ip
```

**Hacer join con una lookup** Ejemplo conceptual: Este patrón permite enriquecer eventos con información externa, como criticidad del activo o propietario.

```
dataset = xdr_data
| filter event_type = NETWORK
| join type=left (
dataset = critical_assets
| fields hostname, asset_criticality, asset_owner
) as lookup agent_hostname = lookup.hostname
| fields _time, agent_hostname, action_remote_ip, lookup.asset_criticality, lookup.asset_ow
```

**Crear una lookup desde resultados de una query** Ejemplo conceptual: Este patrón permite guardar los resultados de una consulta como lookup para reutilizarlos posteriormente.

```
dataset = xdr_data
| filter event_type = PROCESS
| filter actor_process_image_path contains "powershell.exe"
| fields agent_hostname, actor_process_image_path, actor_process_command_line
| target type=lookup append=false powershell_observed
```

### 3.4.6. Stage target y append

El stage **target** permite guardar resultados de una query en un dataset o lookup dataset.

Sintaxis general:

```
| target type=dataset|lookup [append=true|false] nombre_dataset
```

Puntos importantes:

- **target** debe ser el último stage de la query.
- **type=lookup** crea o actualiza una lookup table.
- **append=true** añade los resultados al contenido existente.

- `append=false` reemplaza el contenido existente.
- Si no se indica `append`, el valor por defecto es `false`.

Para el examen:

**Si se usa `target type=lookup` sin `append=true`, se puede reemplazar el contenido existente de la lookup.**

### 3.4.7. TTL de lookup datasets

Las lookup tables pueden tener **TTL**, es decir, un tiempo de vida para sus entradas. Las opciones principales son:

**Forever** Las entradas no expiran. Es el valor por defecto.

**Custom** Las entradas expiran después de un periodo definido en días, horas y minutos.

El TTL se calcula a partir de la última actualización del dato. Las entradas expiradas se eliminan automáticamente mediante un proceso programado.

### 3.4.8. Límites y restricciones

Las lookup tables están pensadas para listas de referencia relativamente pequeñas y de baja frecuencia de cambio.

Puntos importantes:

- Soportan cambios de baja frecuencia (1.200 cambios /día).
- No están pensadas para almacenar grandes volúmenes de eventos.
- Solo un usuario puede editar una lookup en un momento dado.
- Una lookup creada con `target type=lookup` tiene un límite de tamaño (50 MB).
- Una lookup subida desde *Dataset Management* puede tener un límite distinto (30 MB).
- Una lookup dataset no puede eliminarse si existen dependencias, por ejemplo si una *correlation rule* la está usando.
- Para gestionarlas pueden requerirse permisos RBAC sobre *Data Management*.

#### 3.4.9. Uso en detection rules y correlation rules

Las lookup tables pueden utilizarse en reglas de detección o correlation rules. Esto permite que una regla dependa de una lista actualizable sin modificar toda la lógica de la detección.

Ejemplos:

- Detectar logins de usuarios incluidos en una lookup de empleados dados de baja.
- Detectar conexiones hacia IPs incluidas en una lookup de infraestructura sospechosa.
- Detectar ejecución de hashes incluidos en una lookup temporal de indicadores.
- Ignorar actividad procedente de IPs autorizadas mediante una allow list.

También pueden utilizarse acciones como *Add to lookup* para añadir datos a una lookup dataset desde una regla, mapeando campos de la query contra campos del schema de la lookup. Los campos pueden marcarse como únicos para actualizar entradas existentes en lugar de crear duplicados.

#### 3.4.10. Lookup tables vs allow lists y block lists

Una lookup table puede utilizarse para construir allow lists o block lists, pero no son exactamente lo mismo.

**Lookup table** Almacena valores de referencia.

**Allow list** Lógica que permite, ignora o exceptúa valores autorizados.

**Block list** Lógica que bloquea, detecta o prioriza valores no permitidos.“

Ejemplo:

`authorized_admin_ips`

Esta lookup puede contener IPs desde las que los administradores están autorizados a conectarse. La lookup almacena los valores; la regla o query decide qué hacer con ellos.

### 3.5. Identify, hunt, and investigate leads and indicators of compromise (IOCs)

La idea principal es:

Un lead es una pista que requiere investigación; un IOC es un indicador técnico concreto que puede asociarse a compromiso. El analista debe saber buscarlos, validarlos, medir alcance y actuar.

### 3.5.1. Diferencia entre lead e IOC

Un **lead** es una pista de investigación. No tiene por qué ser malicioso por sí mismo, pero puede indicar una actividad que merece análisis.

Ejemplos de leads:

- PowerShell ejecutado con argumentos sospechosos.
- Un usuario autenticándose desde una ubicación inusual.
- Un endpoint comunicándose con una IP poco frecuente.
- Un proceso desconocido apareciendo en varios hosts.
- Un volumen anómalo de conexiones desde un servidor.
- Un hash observado en un endpoint donde no debería aparecer.

Un **IOC**, o *Indicator of Compromise*, es un indicador técnico concreto que puede estar asociado a actividad maliciosa o sospechosa.

Ejemplos de IOCs:

- Hash MD5.
- Hash SHA256.
- Dirección IP de destino.
- Dominio.
- Nombre de fichero.
- Ruta completa de fichero.

La diferencia clave es:

**Lead = pista o hipótesis de investigación.**

**IOC = indicador técnico concreto.**

Ejemplo:

- **Lead:** un usuario ejecutó PowerShell con `EncodedCommand`.
- **IOC:** hash SHA256 del payload descargado.

### 3.5.2. Identificación de leads

Identificar leads significa reconocer señales que pueden iniciar una investigación. Estas señales pueden proceder de issues, cases, búsquedas XQL, alertas analíticas, causality chains, timelines, threat intelligence o resultados de hunting.

Fuentes habituales de leads:

- Issues o cases abiertos.
- Alertas de Analytics.
- Resultados de consultas XQL.
- Causality Chain.
- Timeline.
- Threat intelligence.
- WildFire verdicts.
- Conexiones de red anómalas.
- Actividad de identidad.
- Hunt collections.
- Informes de Managed Threat Hunting, si la organización dispone del servicio.

### 3.5.3. Threat hunting

El **threat hunting** es una búsqueda proactiva de actividad sospechosa. A diferencia de una investigación reactiva, no siempre parte de una issue o alerta ya generada. Puede partir de una hipótesis, un patrón, un IOC o un comportamiento anómalo.

Diferencia clave:

**Investigación reactiva = parte de una issue o case.**

**Threat hunting = parte de una hipótesis, patrón o IOC.**

### 3.5.4. Tipos de IOCs

Los principales tipos de IOC son:

**Full path** Ruta completa de un fichero sospechoso o malicioso.

**File name** Nombre de fichero. Es menos preciso si el nombre es genérico.

**Domain** Dominio asociado a actividad maliciosa o sospechosa.

**Destination IP address** Dirección IP de destino relacionada con una amenaza.

**MD5 hash** Hash MD5 de un fichero.

**SHA256 hash** Hash SHA256 de un fichero. Suele ser un IOC de alta precisión.

### 3.5.5. IOC rules

Una **IOC rule** permite generar detecciones cuando Cortex XDR encuentra coincidencias con indicadores definidos o cargados.

Una IOC rule puede configurarse con:

- Valor del indicador.
- Tipo de IOC.
- Severidad.
- Comentario.
- Reputación.
- Fiabilidad.
- Expiración.

### 3.5.6. Backward scan

El **backward scan** búsqueda histórica (datos ya existentes) de coincidencias de un IOC.

Esto significa que un IOC nuevo no solo se evalúa sobre eventos futuros, sino que también puede buscarse en datos históricos disponibles.

Estados típicos de un backward scan:

- Done.
- Failed.
- Pending.
- Queued.

### 3.5.7. Calidad de un IOC

No todos los IOCs tienen la misma calidad. El analista debe valorar precisión, contexto, reputación, fiabilidad y vigencia.

Ejemplos:

- **SHA256**: indicador muy preciso para identificar un fichero concreto.

- **MD5**: útil, aunque menos robusto que SHA256.
- **File name**: puede generar ruido si es un nombre genérico, como `update.exe`.
- **Destination IP**: puede ser volátil o compartida.
- **Domain**: puede ser útil, pero también puede cambiar de reputación.
- **Full path**: útil si la ruta es específica del comportamiento malicioso.

#### 3.5.8. Expiración de IOCs

Algunos IOCs deben tener expiración, especialmente aquellos que pueden cambiar de significado con el tiempo.

Ejemplos:

- Una IP asociada a una campaña puede ser maliciosa solo durante un periodo corto.
- Un dominio puede ser limpiado o reutilizado.
- Un indicador temporal de campaña puede dejar de ser útil y generar falsos positivos.

#### 3.5.9. IOC vs BIOC vs lead

Es importante diferenciar estos tres conceptos:

**Lead** Pista o hipótesis que requiere investigación.

**IOC** Indicador técnico concreto y normalmente estático.

**BIOC** Indicador de comportamiento sospechoso.

#### 3.5.10. Hunting de IOCs con XQL

XQL permite buscar indicadores en los datasets disponibles para medir alcance e impacto.

##### Buscar hash SHA256

```
dataset = xdr_data
| filter action_file_sha256 = "HASH_SHA256_AQUI" or actor_process_image_sha256 = "HASH_SHA256_AQUI"
| fields _time, agent_hostname, event_type, actor_process_image_path, action_file_path, action_file_sha256
| sort desc _time
| limit 100
```

### Buscar IP sospechosa

```
dataset = xdr_data
| filter action_remote_ip = "1.2.3.4"
| fields _time, agent_hostname, actor_process_image_path, action_remote_ip, action_remote_po
| sort desc _time
| limit 100
```

### 3.5.11. Hunting de leads con XQL

No siempre se dispone de un IOC. A veces el hunting parte de un patrón o comportamiento sospechoso.

### PowerShell con EncodedCommand

```
dataset = xdr_data
| filter actor_process_image_path contains "powershell.exe"
| filter actor_process_command_line contains "EncodedCommand"
| fields _time, agent_hostname, actor_process_image_path, actor_process_command_line
| sort desc _time
| limit 100
```

### Proceso ejecutado desde carpeta temporal

```
dataset = xdr_data
| filter event_type = PROCESS
| filter actor_process_image_path contains "\\Temp\\"
| fields _time, agent_hostname, actor_process_image_path, actor_process_command_line
| sort desc _time
| limit 100
```

## 3.6. Demonstrate understanding of Cortex XDR dashboards and reports

La idea principal es:

**Dashboards** = visualización interactiva para monitorizar actividad.

**Reports** = salida generada o programada para documentar y distribuir información.

**Widgets** = bloques visuales que alimentan dashboards y reports.

### 3.6.1. Dashboard vs Report

Un *dashboard* es una vista visual, normalmente interactiva, que permite monitorizar el estado del entorno. Se utiliza en el día a día del SOC para revisar actividad de casos, issues, agentes, endpoints, ingesta de datos, identidad, cloud, automatización o métricas operativas.

Un *report* es una salida generada a partir de un dashboard o de una plantilla. Sirve para documentar, compartir o entregar información de forma puntual o programada.

La diferencia clave es:

**Dashboard = monitorización visual e interactiva.**

**Report = salida generada o programada para revisión, entrega o documentación.**

Ejemplo:

- Un analista usa un dashboard para ver casos abiertos, agentes no protegidos y endpoints con más issues.
- Un responsable de SOC recibe un report semanal con métricas de casos, MTTR, agentes y tendencias.

### 3.6.2. Componentes principales

**Dashboard** Un dashboard está compuesto por widgets y permite monitorizar actividad del sistema, actividad de seguridad, agentes, casos, datos de ingesta, identidad, cloud o métricas operativas.

Cortex XDR proporciona dashboards predefinidos y también permite crear dashboards personalizados.

**Widget** Un widget es un bloque visual dentro de un dashboard o report. La información se actualiza al momento. Puede mostrar información en distintos formatos, como:

- Tabla.
- Gráfico de líneas.
- Gráfico de columnas.
- Gráfico circular.
- Valor único.
- Texto.

Los widgets pueden ser predefinidos o personalizados.

**Widget Library** La *Widget Library* es el repositorio central de widgets predefinidos y personalizados. Desde ella se pueden buscar, crear, editar y reutilizar widgets.

**Report Template** Un *report template* es una plantilla utilizada para generar reports. Puede crearse desde cero, a partir de una plantilla predefinida o a partir de un dashboard existente.

**Report** Un *report* es el resultado generado desde una report template. Puede generarse bajo demanda o programarse para ejecutarse de forma recurrente.

### 3.6.3. Predefined dashboards

Los *predefined dashboards* son dashboards proporcionados por Palo Alto Networks. Están pensados para monitorizar aspectos concretos del entorno.

Algunos ejemplos son:

- **Agent Management:** estado de agentes desplegados, versiones y sistemas operativos.
- **Case Management:** información sobre casos, hosts y asignaciones.
- **My Dashboard:** visión de casos y MTTR del usuario conectado.
- **Security Admin:** casos, resolución y casos vencidos.
- **Data Ingestion:** ingesta por producto o vendor, cuota y tasa de ingesta.

### 3.6.4. Custom dashboards

Los *custom dashboards* son dashboards personalizados creados por la organización o por el usuario. Sirven para adaptar la visualización al flujo operativo real del SOC.

Pueden crearse:

- Desde cero.
- A partir de un dashboard predefinido.
- A partir de una plantilla existente.

Los custom dashboards pueden compartirse con usuarios o grupos, normalmente con permisos de *Viewer* o *Editor*, si el administrador lo permite. También puede seleccionarse un dashboard disponible como dashboard por defecto al iniciar sesión.

### 3.6.5. Custom XQL widgets

Los *custom XQL widgets* permiten crear widgets basados en consultas XQL. Se utilizan cuando los widgets predefinidos no cubren una necesidad concreta.

Ejemplos de widgets XQL personalizados:

- Top 10 endpoints con más issues críticas.
- Número de conexiones a dominios sospechosos.
- Cases por owner.
- Hosts con agentes en estado unprotected.
- Tendencia diaria de detecciones.

Flujo conceptual:

Crear query XQL

→ seleccionar formato visual

→ guardar widget

→ añadirlo a dashboard o report

Detalle importante:

Las consultas XQL creadas desde la Widget Library para generar custom widgets no aparecen en Query Center; sus resultados se usan para crear el widget.

### 3.6.6. Filters, inputs y drilldowns

Los dashboards y reports pueden refinarse mediante filtros, inputs y drill-downs.

**Filters e inputs** Los filtros e inputs permiten cambiar el alcance de los datos mostrados. Se puede filtrar por endpoint o por severidad.

**Drilldowns** Un drilldown permite hacer clic en un dato de un widget y pivotar hacia más detalle. Puede llevar a:

- Una búsqueda XQL.
- Otro dashboard.
- Un report.
- Una URL personalizada.
- Un cambio contextual dentro del propio dashboard.

Ejemplo:

- Un widget muestra los endpoints con más detecciones.
- El analista hace clic en un endpoint.
- El drilldown abre una búsqueda XQL filtrada por ese endpoint.

**Dashboards** Un usuario puede acceder a:

- Dashboards de los que es owner.
- Dashboards compartidos con él.
- Dashboards compartidos con su grupo.
- Dashboards públicos.

Los predefined dashboards pueden ser públicos para usuarios autorizados, pero los datos mostrados dependen del scope de datos del usuario.

**Widgets** Los widgets pueden ser públicos o restringidos.

**Public widget** Puede ser visible para usuarios con acceso a la Widget Library.

**Restricted widget** Visible normalmente para su creador en la Widget Library.

Si un dashboard compartido contiene un widget restringido, los usuarios autorizados a ver ese dashboard pueden ver los datos del widget en ese dashboard, aunque no vean el widget en su propia Widget Library.

También debe recordarse que modificar un widget puede afectar a los dashboards y reports que lo utilizan.

### 3.6.7. Importación y exportación

Cortex XDR permite importar y exportar dashboards y report templates en formato JSON para mover configuraciones entre entornos.

### 3.6.8. Información que suelen mostrar dashboards y reports

Dashboards y reports pueden mostrar información como:

- Cases abiertos.
- Cases resueltos.
- Cases críticos.
- Cases vencidos.
- MTTR.
- Cases por assignee.

- Cases por severidad.
- Top affected hosts.
- Estado de agentes.
- Agentes por sistema operativo.
- Versiones de agentes y contenido.
- Endpoints protected, partially protected o unprotected.
- Tendencias de detección.
- Uso de automatización.
- Data ingestion por vendor o product.
- Cuota de ingesta.
- Actividad de identidad.
- Cloud posture o cloud security operations, si aplica licencia.

La finalidad no es solo visualizar datos, sino convertir datos operativos y de seguridad en métricas accionables.

### 3.7. Identify and explain the data retention options in Cortex XDR

#### 3.7.1. ¿Qué es data retention en Cortex XDR?

Cortex XDR almacena datos en el **Cortex XDR Data Layer**. La retención depende de la licencia contratada, del tipo de dato y de los add-ons de retención. Palo Alto indica que todas las licencias proporcionan periodos de retención por defecto, y que estos pueden ampliarse con opciones de hot storage y cold storage.

El flujo general cuando llegan los datos es:

1. Data Ingestion Pipeline
2. Normalization / enrichment / analytics
3. Hot storage / Cold storage / Export

Si los datos se purgan, se eliminan para siempre.

Los datos entran por el **Data Ingestion Pipeline**, donde pueden normalizarse, enriquecerse y usarse para analítica; después pasan a los destinos de almacenamiento según la licencia y los add-ons disponibles.

En Cortex XDR se distinguen varios tipos de datos:

- **Ingested data:** logs y telemetría ingerida desde endpoints, red, cloud o terceros

- **Case and Issue data:** información de cases e issues, como estado, severidad, evidencias, assets, artifacts, etc.
- **Forensic data:** datos recogidos por el módulo/add-on de Forensics
- **Query data:** datos disponibles para consultas e investigación mediante XQL
- **Export/Event Forwarding data:** datos reenviados fuera de Cortex XDR mediante capacidades de exportación

Los **ingested data** tienen retención base de **31 días** en licencias con ingesta, que los **Case and Issue data** se conservan durante **186 días** en los modelos donde aplica, que los **forensic data** se conservan **365 días** si se dispone del add-on de Forensics, y que los **query data** tienen retención de **186 días** en los modelos indicados.

Los Case and Issue data se conservan durante 186 días en los modelos donde aplica. Además, Palo Alto indica que los datos de case e issue se retienen según la fecha de última actualización y creación, respectivamente. Para mantener la precisión de los cases, Cortex XDR proporciona un periodo de gracia de hasta 31 días para issues mostradas en la vista de cases, tabla de issues y Causality View.

### 3.7.2. Hot storage

**Hot storage** es el almacenamiento activo y plenamente consultable. Es el tipo de almacenamiento que se usa para investigación diaria, threat hunting, dashboards, XQL y análisis operativo.

La documentación indica que, con una licencia regular, los datos se envían automáticamente a hot storage durante el periodo de retención por defecto, normalmente un mes; ese periodo puede ampliarse mediante retención basada en periodo o almacenamiento hot adicional.

Ejemplo de búsqueda XQL sobre un host storage:

```
dataset = xdr_data
| filter event_type = "PROCESS"
```

### 3.7.3. Cold storage

**Cold storage** es almacenamiento de **menor coste** orientado a **retención a largo plazo**, normalmente por requisitos de cumplimiento, auditoría o investigación histórica. La documentación lo describe como una opción de almacenamiento más barata para necesidades de compliance a largo plazo, pero con opciones de búsqueda limitadas

Ejemplo de consulta XQL sobre cold storage:

```
cold_dataset = xdr_data
| filter event_type = "PROCESS"
```

| Característica | Hot storage                           | Cold storage                                     |
|----------------|---------------------------------------|--------------------------------------------------|
| Objetivo       | Investigación activa y threat hunting | Retención histórica y compliance                 |
| Coste          | Mayor                                 | Menor                                            |
| Búsqueda       | Fully searchable                      | Búsqueda limitada                                |
| XQL            | <code>dataset = xdr_data</code>       | <code>cold_dataset = xdr_data</code>             |
| Uso típico     | SOC diario, hunting, dashboards       | Auditoría, cumplimiento, investigación histórica |
| Disponibilidad | Durante el periodo hot                | Tras expirar el periodo hot                      |
| Requiere CU    | No como cold                          | Sí, para consultas cold                          |

Figura 7: Hot vs Cold storage.

Las consultas en cold storage consumen **Compute Units**, o **CU**. Palo Alto indica que las queries sobre cold storage consumen CU en función de la cantidad de datos consultada, el rango temporal, la complejidad y el número de resultados. También indica una referencia: 1 CU por 35 GB consultados.

#### 3.7.4. Retention add-ons

Cortex XDR permite ampliar la retención mediante add-ons. La documentación actual lista tres ideas principales:

- Primero, **Additional 31-day hot storage** of Case and Issue data. Sirve para añadir 31 días adicionales de hot storage para datos de cases e issues, según licencia.
- Segundo, **Period-based retention - hot storage**. Es almacenamiento fully searchable para investigación y threat hunting de ingested data y Case/Issue data. Requiere comprar al menos 1 mes de retención adicional en los modelos donde aplica.
- Tercero, **Period-based retention - cold storage**. Es almacenamiento de menor coste para ingested data orientado a compliance de largo plazo, con opciones de búsqueda limitadas, y requiere comprar al menos 6 meses de retención adicional.

#### TTL en lookup datasets

Los lookup datasets pueden tener un TTL, es decir, un tiempo de vida para que las entradas expiren y se eliminen automáticamente. La documentación indica que, para lookup datasets, el campo TTL muestra cuándo expiran las entradas y que el valor por defecto puede ser **Forever**, es decir, que las entradas no expiran.

### 3.7.5. Retención y pérdida de datos

Palo Alto recomienda gestionar la retención de forma proactiva. Si una licencia expira o el periodo de retención pasa, los datos se purgan y no pueden recuperarse ni extenderse retroactivamente.

**La idea clave:** Solo puedes ampliar retención retroactivamente sobre datos que todavía existan. Si los datos ya fueron purgados, no se pueden recuperar.

## 3.8. Explain the use of Host Insights information

Host Insights es una capacidad de Cortex XDR que permite obtener contexto técnico de un endpoint durante una investigación. Su finalidad no es generar una alerta por sí misma, sino proporcionar información de inventario y artefactos del host para ayudar al analista a entender mejor el estado del endpoint afectado.

La idea clave para el examen es:

**Host Insights = información de inventario y artefactos del endpoint que ayuda a entender el contexto, el alcance y posibles elementos sospechosos en un host.**

### 3.8.1. Finalidad de Host Insights

Host Insights se utiliza para enriquecer la investigación de un host afectado. Permite revisar información como aplicaciones instaladas, autoruns, servicios, drivers, usuarios, grupos, shares, discos e información general del sistema.

### 3.8.2. Dónde se consulta en Cortex XDR

El flujo habitual durante una investigación es:

**Incidente o alerta → host afectado → Open Asset View → Host Insights**

En la *Asset View*, el panel izquierdo muestra información general del host y los incidentes relacionados. En el panel derecho se puede seleccionar el tipo de información que se quiere visualizar.

Las opciones principales son:

- **Host Insights:** muestra artefactos e información de inventario del host.
- **Network Connections:** permite pivotar hacia las direcciones IP asociadas al host.
- **Host Risk View:** muestra información de riesgo y perfilado, disponible con Identity Threat Module.

### 3.8.3. Relación entre Host Insights y Host Inventory

Host Insights y Host Inventory están relacionados, pero no son exactamente lo mismo.

- **Host Insights:** se centra en el contexto de un host concreto durante una investigación.
- **Host Inventory:** proporciona una visión global de inventario sobre todos los endpoints.

La ruta habitual para acceder a Host Inventory es:

**Incident Response → Investigation → Host Inventory**

### 3.8.4. Información que puede mostrar Host Insights

Host Insights puede incluir diferentes tipos de información, dependiendo del sistema operativo del endpoint.

Las categorías más relevantes son:

- **Applications:** aplicaciones instaladas, incluyendo posibles CVEs y severidad de vulnerabilidades.
- **Autoruns:** ejecutables configurados para arrancar automáticamente.
- **Services:** servicios presentes en el endpoint, estado, usuario de ejecución y modo de inicio.
- **Drivers:** drivers instalados, ruta, tipo y estado de ejecución.
- **Daemons:** procesos o servicios persistentes en sistemas macOS o Linux.
- **Disks:** volúmenes, sistema de ficheros, espacio disponible y tamaño total.
- **Shares:** carpetas compartidas de red.
- **System Information:** fabricante, modelo, memoria, arquitectura, CPU, sistema operativo y versión.
- **Users:** usuarios locales y propiedades de sus cuentas.
- **Groups:** grupos definidos en el endpoint.
- **Users to Groups:** relación entre usuarios y grupos.

### 3.8.5. Recogida de información

Host Insights requiere que la capacidad esté habilitada y que el entorno disponga de la licencia o add-on correspondiente.

De forma general, para utilizar esta funcionalidad se necesita:

- Cortex XDR Pro per Endpoint.
- Host Insights Add-on.
- Host Inventory Data Collection habilitado en el agente.
- Endpoint con agente de Cortex XDR instalado.

El agente de Cortex XDR recopila periódicamente información del endpoint. Host Inventory puede escanear el endpoint cada 24 horas, mostrar información de los últimos 30 días y permitir un rescán para obtener datos más actualizados.

También se puede usar la opción *Run insights collection* desde la Asset View. En ese caso, la información se recopila la siguiente vez que el agente conecta con Cortex XDR.

## 3.9. Diferencia con Operational Status

Host Insights tampoco debe confundirse con el estado operativo del agente.

- **Host Insights:** información de inventario y contexto del endpoint.
- **Operational Status:** indica si el agente está protegiendo correctamente el endpoint.

Los estados principales del agente son:

- **Protected:** el agente funciona según la configuración esperada.
- **Partially protected:** el agente ha reportado una o más excepciones.
- **Unprotected:** el agente no está aplicando protección en el endpoint.
- **Local Resource Impact:** el endpoint no tiene recursos locales suficientes para que el agente funcione correctamente.

## 3.10. Diferencia entre Endpoint Status y Operational Status

En Cortex XDR es importante diferenciar dos conceptos:

- **Endpoint Status:** indica el estado de conexión o registro del agente con Cortex XDR.
- **Operational Status:** indica si el agente está protegiendo correctamente el endpoint.

Los estados principales son:

- **Connected**
- **Disconnected**
- **Connection Lost**
- **VDI Pending Log-on**
- **Uninstalled**

**Connected** = el agente se está comunicando recientemente con Cortex XDR.

En endpoints estándar, se considera conectado si ha hecho *check-in* dentro de los últimos 10 minutos. En endpoints móviles, dentro de las últimas 3 horas.

Es importante recordar que **Connected** no significa necesariamente **Protected**. Un agente puede estar conectado, pero tener problemas de protección. Ejemplo:

**Endpoint Status: Connected**  
**Operational Status: Unprotected**

Esto significa que el agente comunica con Cortex XDR, pero no está aplicando protección correctamente.

**Disconnected** = el agente no se está comunicando actualmente con Cortex XDR dentro del intervalo esperado.

**Connection Lost** = el agente lleva demasiado tiempo sin hacer check-in con Cortex XDR.

El estado **Uninstalled** indica que el agente ha sido desinstalado del endpoint.

## 4. Endpoint Security Management (15 %)

### 4.1. Demonstrate understanding of endpoint prevention and extension profiles and policies

En Cortex XDR, la gestión de seguridad del endpoint se basa en perfiles, políticas y grupos de endpoints. Los **perfiles** definen el comportamiento de protección del agente, mientras que las **políticas** determinan a qué endpoints o grupos se aplican esos perfiles.

La idea clave para el examen es:

**Profiles define behavior; policies assign that behavior to endpoints.**

#### 4.1.1. Componentes principales

Los componentes principales son:

- **Security profiles:** objetos reutilizables donde se configura cómo debe comportarse el agente.
- **Policy rules:** reglas que asignan perfiles a endpoints o grupos de endpoints.
- **Endpoint groups:** grupos de endpoints usados como destino de las políticas.
- **Exceptions:** reglas o perfiles que modifican el comportamiento de prevención en casos concretos.

La lógica general es:

**Perfil → Política → Endpoint o grupo de endpoints**

#### 4.1.2. Ruta en la interfaz

La ruta principal en Cortex XDR es:

**Endpoints → Policy Management → Prevention**

Dentro de esta sección se utilizan principalmente:

- **Profiles:** para crear o editar perfiles.
- **Policy Rules:** para aplicar perfiles a endpoints o grupos.

#### 4.1.3. Policy rules

Una *policy rule* permite asignar perfiles de seguridad a endpoints concretos o grupos de endpoints.

Al crear una policy rule se define:

- Nombre de la política.
- Plataforma: Windows, macOS, Linux u otra plataforma soportada.
- Perfil de Exploit.
- Perfil de Malware.
- Perfil de Restrictions.
- Perfil de Agent Settings.
- Targets: endpoints o endpoint groups.

Si no se selecciona un perfil específico, Cortex XDR aplica el perfil por defecto.

#### 4.1.4. Orden de evaluación de políticas

Las políticas se evalúan de arriba hacia abajo. Cuando el agente encuentra la primera política que coincide con el endpoint, aplica esa política como política activa.

Para examen:

**Policy rules are evaluated top-down; first match wins.**

Esto implica que las políticas más específicas deben situarse por encima de las políticas generales.

Ejemplo:

- Política 1: servidores críticos.
- Política 2: todos los endpoints Windows.

Si un servidor crítico también cumple la política general de Windows, la política de servidores críticos debe estar por encima.

#### 4.1.5. Tipos principales de perfiles

Los perfiles principales que se deben conocer son:

- **Malware Prevention Profile**
- **Exploit Prevention Profile**
- **Restrictions Prevention Profile**
- **Agent Settings Profile**
- **Exception Profile / Exception Rules**

#### 4.1.6. Malware Prevention Profile

El **Malware Prevention Profile** controla qué acciones realiza el agente cuando detecta malware conocido, macros, scripts, ransomware, archivos desconocidos o comportamientos asociados a malware.

Muchas capacidades de prevención utilizan tres modos principales:

- **Block:** detecta y bloquea la acción o ejecución maliciosa.
- **Report:** detecta y reporta el evento, pero permite la ejecución, es decir, no bloquea.
- **Disabled:** no analiza, no bloquea y no reporta.

Ejemplo típico:

Si un módulo de Malware Prevention detecta malware, pero está configurado en **Report mode**, Cortex XDR genera una alerta o reporte, pero no bloquea la ejecución.

#### 4.1.7. WildFire y Local Analysis

Cuando un archivo es desconocido, Cortex XDR puede tratarlo de varias formas:

- **Allow:** permite la ejecución.
- **Run Local Analysis:** el agente usa machine learning local para emitir un veredicto.
- **Block:** bloquea el archivo desconocido hasta recibir un veredicto oficial.

Además, si está habilitado, Cortex XDR puede enviar archivos desconocidos a WildFire para análisis cloud (hasta 300 MB).

Para examen:

**WildFire = análisis cloud. Local Analysis = análisis local mediante machine learning del agente.**

#### 4.1.8. Exploit Prevention Profile

El **Exploit Prevention Profile** controla qué hace el agente cuando se producen intentos de explotar vulnerabilidades o fallos de software.

Este perfil protege frente a:

- Exploits de navegador.
- Exploits del sistema operativo.
- Exploits contra procesos vulnerables.
- Exploit kits.
- Ejecución ilegal de código.
- Técnicas de corrupción de memoria.
- Manipulación del flujo de ejecución.
- Abuso de procesos protegidos.

Los *protected processes* son procesos sobre los que Cortex XDR aplica módulos de exploit prevention.

Pueden incluir:

- Navegadores.
- Lectores PDF.
- Aplicaciones Office.
- Procesos del sistema operativo.

- Aplicaciones de terceros añadidas manualmente.
- Aplicaciones internas o propietarias.

La política por defecto protege las aplicaciones más vulnerables y usadas habitualmente, pero se pueden añadir procesos adicionales.

#### 4.1.9. Restrictions Prevention Profile

El **Restrictions Prevention Profile** limita desde qué ubicaciones pueden ejecutarse ejecutables en un endpoint.

Sirve para reducir la ejecución desde ubicaciones sospechosas o no habituales, como:

- Carpetas temporales.
- Directorios de usuario.
- Rutas de descarga.
- Ubicaciones no autorizadas.

No debe confundirse con Malware Prevention. Un archivo puede no estar clasificado como malware y aun así estar bloqueado o limitado por ejecutarse desde una ubicación restringida.

#### 4.1.10. Agent Settings Profile

El **Agent Settings Profile** configura el comportamiento general del agente Cortex XDR según plataforma y grupo de endpoints.

Puede incluir opciones relacionadas con:

- Configuración general del agente.
- Tamper Protection.
- Notificaciones al usuario.
- Recogida de datos.
- Configuraciones específicas por sistema operativo.
- Opciones de actualización.
- Funciones adicionales del agente.

#### 4.1.11. Exception profiles y exception rules

Las **exceptions** permiten modificar el comportamiento de protección cuando una actividad legítima está siendo bloqueada o interferida por el agente.

No son lo mismo que las alert exclusions.

- **Alert exclusion:** oculta o suprime alertas futuras que coinciden con ciertos criterios.
- **Exception:** modifica el comportamiento de una regla, módulo, perfil o política de protección.

#### 4.1.12. Diferencia entre profile y policy

Un **profile** define la configuración.

Una **policy** decide a qué endpoints se aplica esa configuración.

Ejemplo:

- Malware Prevention Profile: bloquear ransomware y subir archivos desconocidos a WildFire.
- Policy Rule: aplicar ese perfil a servidores Windows de producción.

### 4.2. Identify and validate the impact of agent operational states

Este apartado evalúa si el analista sabe interpretar el estado operativo del agente de Cortex XDR y valorar qué impacto real tiene sobre la protección, la visibilidad y la capacidad de respuesta del endpoint.

La idea clave para el examen es:

**No basta con ver que un agente aparece en la consola.  
Hay que comprobar si está protegiendo correctamente, qué  
módulo falla y qué impacto tiene ese fallo.**

El **Endpoint Status** responde a la pregunta:

**¿El agente se comunica con Cortex XDR?**

El **Operational Status** responde a la pregunta:

**¿El agente protege correctamente el endpoint?**

Por tanto, un agente puede estar **Connected** y aun así estar **Unprotected**.

#### 4.2.1. Operational Status

Los estados principales son:

- **Protected**
- **Partially protected**
- **Unprotected**
- **Local Resource Impact**

El estado **Protected** indica que el agente está ejecutándose según la configuración esperada y no ha reportado excepciones.

En Windows, macOS y Linux, este estado implica que los módulos de protección están funcionando según la configuración aplicada al endpoint.

Impacto:

- El endpoint tiene la protección esperada.
- Los módulos configurados están activos.
- Las políticas y perfiles aplicados funcionan correctamente.
- Es el estado deseable.

El estado **Partially protected** indica que el agente está activo, pero ha reportado una o más excepciones a Cortex XDR.

No significa que el endpoint esté totalmente desprotegido. Significa que una capacidad concreta no está funcionando correctamente.

Causas típicas en Windows:

- XDR Data Collection no está funcionando o no está configurado.
- Behavioral Threat Protection no está funcionando.
- Malware Protection no está funcionando.
- Exploit Protection no está funcionando.

Causas típicas en macOS:

- Operating System adaptive mode.
- XDR Data Collection no está funcionando.
- Behavioral Threat Protection no está funcionando.
- Malware Protection no está funcionando.
- Exploit Protection no está funcionando.

Causas típicas en Linux:

- Kernel module not loaded.
- Kernel module compatible but not loaded.
- Kernel version not compatible.
- XDR Data Collection no está funcionando.
- Behavioral Threat Protection no está funcionando.
- Anti-malware flow is asynchronous.
- Malware Protection no está funcionando.
- Exploit Protection no está funcionando.

El estado **Unprotected** indica que el agente no está aplicando protección sobre el endpoint.

**Unprotected = no protege. Connected no significa Protected.**

Ejemplo:

**Endpoint Status: Connected**  
**Operational Status: Unprotected**

Interpretación:

El agente se comunica con Cortex XDR, pero no está aplicando protección correctamente.

El estado **Local Resource Impact** indica que el endpoint no dispone de recursos locales suficientes para que el agente funcione correctamente.

Causas típicas:

- Impacto de CPU sobre la operación del agente.
- Impacto de memoria sobre la operación del agente.
- Low local available memory.
- No local available memory.
- Si el endpoint está **Protected**, se espera telemetría y protección según configuración.
- Si está **Partially protected** por fallo de XDR Data Collection, puede haber menos eventos para investigar.
- Si está **Disconnected**, no hay telemetría actual.

- Si está **Unprotected**, la ausencia de alertas no garantiza ausencia de actividad maliciosa.

Frase clave:

**Un endpoint sin alertas, pero Unprotected o sin data collection, no debe considerarse necesariamente limpio.**

### 4.3. Identify and validate the impact of agent version and content update

Este apartado evalúa si el analista sabe comprobar si un endpoint tiene una versión adecuada del agente de Cortex XDR y si el contenido de seguridad está actualizado.

La idea clave para examen es:

**Un agente puede estar instalado y conectado, pero si está desactualizado o con contenido antiguo, puede tener menos capacidades de prevención, peor compatibilidad o menor cobertura frente a amenazas recientes.**

#### 4.3.1. Diferencia entre Agent Version y Content Version

**Agent Version** indica la versión del software del agente instalada en el endpoint.

**Content Version** indica la versión del contenido de seguridad que utiliza el agente.

Para examen:

**Agent Version = motor/software del agente.**

**Content Version = inteligencia, reglas y lógica que usa el agente.**

Un endpoint puede tener una versión reciente del agente, pero una versión antigua de contenido. También puede ocurrir lo contrario: contenido actualizado sobre una versión de agente que ya no es la más adecuada.

#### 4.3.2. Dónde se valida en la interfaz

La ruta principal es:

**Endpoints → All Endpoints**

Desde esa tabla deben revisarse columnas como:

- **Agent Version**
- **Content Auto Update**

- **Content Release Timestamp**
- **Content Rollout Delay**
- **Content Status**
- **Content Version**
- **Last Content Update Time**
- **Last Upgrade Status**
- **Last Upgrade Failure Reason**
- **Last Upgrade Source**
- **Last Upgrade Status Time**
- **Endpoint Status**
- **Operational Status**

#### 4.3.3. Agent Version

La **Agent Version** afecta a las capacidades disponibles del agente.

#### 4.3.4. Last Upgrade Status

El campo **Last Upgrade Status** puede mostrar:

- **In Progress**: el upgrade está en curso.
- **Completed Successfully**: el upgrade terminó correctamente.
- **Failed**: el upgrade falló.
- **No Upgrade**: no se ha iniciado ningún upgrade para ese endpoint.

Para examen:

**Si Last Upgrade Status = Failed, hay que revisar Last Upgrade Failure Reason.**

#### 4.3.5. Auto Upgrade y rollout

La actualización del agente puede gestionarse mediante auto-upgrade.

**Agent Auto-Upgrade** permite actualizar automáticamente los agentes dentro del alcance definido por la organización.

El rollout puede ser:

- **Immediate**: el agente recibe automáticamente nuevas versiones.
- **Delayed**: el upgrade se retrasa durante un periodo configurado.

#### 4.3.6. Restricciones importantes de upgrade

No todos los endpoints se actualizan igual.  
Hay que tener especial cuidado con:

- **Mobile**
- **VDI**
- **TS**
- **Golden Image**

Para examen:

**No se debe asumir que cualquier endpoint puede usar auto-upgrade.**

#### 4.3.7. Content updates

Los **content updates** son paquetes de contenido de seguridad que recibe el agente.

Pueden incluir cambios o actualizaciones en:

- Política de seguridad por defecto.
- Reglas de compatibilidad por módulo.
- Procesos protegidos.
- Lógica de Local Analysis.
- Trusted signers.
- Procesos incluidos en block lists por signers.
- Reglas de Behavioral Threat Protection.
- Lógica del módulo de ransomware.
- Event logs para Windows y Linux.
- Scripts de Palo Alto Networks.
- Módulos Python para script execution.
- Reglas del Network Packet Inspection Engine.

#### 4.3.8. Content Status

El campo **Content Status** indica el estado de la versión de contenido en el endpoint.

Estados principales:

- **Up to Date:** el endpoint ejecuta la última versión de contenido.
- **Waiting for Update:** Cortex XDR está en proceso de actualizar el contenido.
- **Outdated:** el endpoint utiliza una versión de contenido antigua.
- **Offline:** el endpoint está desconectado.

#### 4.3.9. Content Auto Update

**Content Auto Update** indica si las actualizaciones automáticas de contenido están habilitadas o deshabilitadas para el endpoint.

Si está habilitado, el agente puede recuperar automáticamente contenido nuevo.

Si está deshabilitado, el agente sigue funcionando con el contenido existente, pero no obtiene nuevos contenidos automáticamente.

#### 4.3.10. Major y minor content updates

Las actualizaciones de contenido pueden ser:

- **Major releases:** versiones principales.
- **Minor releases:** versiones intermedias más frecuentes.

Las minor releases permiten una protección más rápida frente a nuevas amenazas.

#### 4.3.11. Last Content Update Time

**Last Content Update Time** indica cuándo desplegó el agente por última vez una actualización de contenido. Este campo permite medir la antigüedad real del contenido. Ejemplo:

**Endpoint Status:** Connected  
**Content Status:** Outdated  
**Last Content Update Time:** hace 20 días

#### 4.3.12. Casos típicos de examen

**Caso 1: Agent Version antigua + Content Status Up to Date**

El contenido está actualizado, pero el software del agente puede no tener las últimas capacidades o compatibilidad.

**Caso 2: Agent Version actual + Content Status Outdated**

El software está actualizado, pero la lógica de seguridad no. Hay menor cobertura frente a amenazas recientes.

**Caso 3: Content Status Waiting for Update**

No necesariamente es un fallo. Cortex XDR está en proceso de actualizar el contenido.

**Caso 4: Content Status Offline**

El problema principal es la conectividad del endpoint. Si está offline, no puede recibir contenido ni enviar telemetría actual.

**Caso 5: Last Upgrade Status Failed**

El upgrade falló. Hay que revisar **Last Upgrade Failure Reason**.

**Caso 6: Content Auto Update Disabled**

El agente no descargará nuevos contenidos automáticamente. Se queda con el contenido existente.

| Dataset / Preset      | Para qué sirve                                                                                                                                                     |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| xdr_data              | Dataset principal de Cortex XDR. Contiene eventos de endpoint, red y logs ingeridos desde distintas fuentes. Es el dataset más usado para hunting e investigación. |
| alerts                | Permite consultar alertas generadas en Cortex XDR, incluyendo nombre de alerta, severidad, host afectado, estado, etc.                                             |
| incidents             | Permite consultar incidentes de Cortex XDR, su severidad, estado, alertas asociadas y otra información de investigación.                                           |
| endpoints             | Contiene información sobre endpoints gestionados por Cortex XDR, como hostname, sistema operativo, versión del agente, estado y grupo de endpoints.                |
| host_inventory        | Contiene información de inventario del host: hardware, software instalado, usuarios, servicios, discos y otros datos del endpoint.                                 |
| auth_logs             | Dataset optimizado para logs de autenticación. Suele usarse para investigar inicios de sesión, autenticaciones sospechosas y actividad de identidad.               |
| login_logs            | Dataset optimizado para eventos de login, por ejemplo eventos de inicio de sesión procedentes de Windows Event Collector u otras fuentes.                          |
| vpn_logs              | Dataset optimizado para eventos VPN, por ejemplo conexiones GlobalProtect u otras fuentes VPN integradas.                                                          |
| microsoft_windows_raw | Contiene logs raw de Windows, como eventos de seguridad, sistema o aplicación ingeridos desde Windows Event Collector o desde el agente.                           |
| panw_ngfw_raw         | Datasets raw procedentes de firewalls Palo Alto Networks. Incluye logs como traffic, threat, URL, auth, GlobalProtect, system, entre otros.                        |
| cloud_audit_logs      | Contiene logs de auditoría cloud normalizados o enriquecidos, por ejemplo de AWS, Azure o GCP, según las integraciones configuradas.                               |
| va_cves               | Contiene información de vulnerabilidades y CVEs detectadas, si está habilitado Host Inventory and Vulnerability Assessment.                                        |
| va_endpoints          | Relaciona endpoints con información de vulnerabilidades detectadas en ellos.                                                                                       |
| parsing_rules_errors  | Contiene errores relacionados con parsing rules. Es útil para troubleshooting de ingesta y normalización de logs.                                                  |
| correlations_auditing | Contiene información de auditoría sobre ejecuciones de correlation rules.                                                                                          |
| collection_auditing   | Contiene información sobre cambios de estado de collectors e integraciones de colección de datos.                                                                  |
| host_firewall_events  | Contiene eventos relacionados con el enforcement del Host Firewall de Cortex XDR.                                                                                  |
| authentication_story  | Preset orientado a analizar actividad de autenticación de forma normalizada. No es exactamente un dataset raw, sino una vista útil sobre datos de autenticación.   |
| network_story         | Preset orientado a analizar actividad de red de forma normalizada. Es útil para investigar conexiones, IPs, puertos, protocolos y actividad de red.                |

Cuadro 1: Principales datasets y presets en Cortex XDR

## 5. Preguntas interesantes

**Cual es el número máximo de alertas para deshabilitar un IOC detection rule?**

El límite es 5.000 hits en 24 horas.

**En que casos Cortex XDR dejaría ejecutar un malware?**

- **Modo Report / Notify:** detecta o informa, pero no bloquea.
- **Unknown file en Allow:** wildFire aún no lo conoce y la política permite desconocidos.
- **Hash exception / allow list:** se fuerza un tratamiento benigno o permitido.
- **Disable Prevention Rule:** se omiten acciones de prevención para ese scope.
- **Agent Unprotected / Partially protected:** el agente no aplica toda la protección.
- **Módulo no activo/no soportado:** la técnica o tipo de archivo no está cubierto.
- **WildFire error + Local Analysis no disponible:** no hay veredicto efectivo para bloquear.

**¿Qué son los reportes de WildFire?**

WildFire report = informe técnico que explica qué hizo una muestra sospechosa y qué veredicto recibió.

WildFire analiza muestras en un entorno de sandbox/análisis avanzado y genera un veredicto. Los veredictos típicos son:

- Benign: benigno
- Grayware: software no claramente malicioso, pero potencialmente no deseado
- Malware: malicioso
- Phishing: enlace o sitio de phishing

**Dime que son las Remediation suggestions**

Las Remediation Suggestions son recomendaciones de acciones correctivas que Cortex XDR genera para ayudarte a revertir cambios producidos por una actividad maliciosa. Ejemplos de acciones sugeridas:

- Delete File
- Restore File
- Rename File
- Delete Registry Value

- Restore Registry Value
- Terminate Process
- Terminate Causality
- Manual Remediation

**Alerta Restriction profile, Acción "Detected", ejecutable que debería de ser bloqueado por profile settings exception configuration de porque se ejecuta?**

Se ejecuta porque “Detected” no significa “Blocked”. En una alerta de Restrictions Profile, si la acción aparece como Detected, Cortex XDR ha detectado que el ejecutable coincide con una condición de restricción, pero no lo ha bloqueado

#### **¿Qué son los custom mappers y los classifiers?**

Los custom classifiers y custom mappers son mecanismos de clasificación y normalización de datos entrantes desde una integración, sobre todo en el contexto de Cortex XSOAR / Cortex platform, cuando Cortex recibe alertas, eventos, incidentes o indicadores desde herramientas externas.

La diferencia clave es:

**Classifier** = decide qué tipo de incidente es.

**Mapper** = rellena los campos del incidente.

Un classifier determina cómo se debe clasificar un evento que entra desde una integración. Es decir, decide qué incident type se va a crear en Cortex. Palo Alto lo define como el mecanismo que determina el tipo de incidente creado para eventos ingeridos desde una integración. Esto permite que eventos distintos usen playbooks distintos, layouts distintos y procesos de respuesta diferentes.

Un mapper, en cambio, no decide el tipo de incidente. Su función es mapear los campos del evento entrante a los campos internos de Cortex. Palo Alto indica que el mapping permite llevar información importante de alertas entrantes a campos de incidente, para que puedan usarse en playbooks, layouts, búsquedas, correlación, deduplicación, reporting e investigación.

#### **¿Qué son las parsing rules?**

Las Parsing Rules en Cortex XDR son reglas usadas en el proceso de ingesta de datos para convertir logs externos o de terceros desde formato bruto a datos estructurados y consultables.

#### **¿Qué es XDR built-in ML and Artificial intelligence driven correlation engine?**

En Cortex XDR, built-in ML and Artificial Intelligence-driven correlation engine se refiere a la capa interna de análisis que usa machine learning, analítica de comportamiento y correlación automática para detectar actividad sospechosa y agrupar eventos relacionados.

#### **Dime como puedo comentar en una query XQL y tambien que es la tamper protection**

Para un comentario de una sola línea usas `//` y `/*multi-line <comments>*/` para comentarios multilínea.

**Tamper Protection:** es la protección que evita que un usuario local, malware o atacante manipule el agente de Cortex XDR en el endpoint.

### **Diferencia entre data models y parsing rules**

Un **data model** define cómo deben organizarse los datos: qué campos existen, qué significan, cómo se llaman y cómo se consultan. Piensa en él como el “molde” o esquema común. Por ejemplo, que una IP origen se represente en un campo concreto, que un usuario tenga un campo concreto, que un proceso tenga ruta, hash, command line, etc.

Una **parsing rule**, en cambio, trabaja antes o durante la ingesta. Coge un log bruto, lo interpreta, extrae campos, los transforma, los normaliza y decide qué filas se insertan en Cortex XDR. Palo Alto describe las parsing rules como reglas que toman raw logs, aplican transformaciones mediante XQL y devuelven cero, una o varias filas que se insertan en el tenant.

### **¿Los logs de los agentes y de las terceras partes donde se almacenan?**

Los logs generados por los Cortex XDR Agents se envían a la infraestructura cloud de Cortex. Palo Alto indica que los agentes de Cortex XDR y Cortex XDR envían los logs al **Cortex Native Data Lake**, y que esos logs pueden verse desde Cortex XDR.

### **What is the Wildfire analysis file size limit for Windows PE files?**

100 MBs.

### **Which dataset does Cortex XDR query to retrieve the operating system version, agent version, and last seen time of a host?**

endpoint\_inventory

### **Which best describes the relationship between Cortex XDR Pro per Endpoint and Cortex XDR Pro per TB?**

Pro per Endpoint is licensed by the number of protected endpoints and unlocks EDR/XDR capability on those agents. Pro per TB is licensed by ingest volume of third-party (non-Palo) logs into the data lake.

### **¿Las issues low generar cases? ¿Aparecen en la tabla de issues?**

### **¿Qué relación tienen las issues low y las correlation rules?**

- Una Issue Low sí existe como issue y aparece en la Issues Table.
- Una Issue Low, por regla general, no genera un Case. Se queda como una detección/Insight para investigación o correlación. En el dominio Security, Cortex promociona normalmente a Case las issues Medium, High y Critical, aunque Palo Alto indica que determinadas analíticas Low internas pueden ser excepciones.
- Las Correlation Rules generan Issues. Cuando creas una correlation rule eliges la severidad de la issue que producirá: Informational, Low, Medium, High o Critical. Esas issues se pueden consultar en la Issues Table.
- Y aquí está lo importante para examen: una Low Issue generada por una Correlation Rule NO se agrupa en un Case. Si la correlation rule genera una issue Medium o superior, entonces se abre automáticamente un Case. Palo Alto lo documenta expresamente.

Ejemplo: haces una correlation rule que detecta 5 failed logins + successful login. Si configuras la regla como Low, cuando se cumpla la condición aparecerá una

Low Issue en Issues, pero no te abre Case. Si la configuras como Medium, esa Issue sí provocará la creación de un Case.

**Donde se encuentra el user/host risk view**

El User/Host Risk View pertenece al módulo Identity Threat Detection & Response (ITDR) y requiere ese add-on.

En la interfaz actual:

- Host Risk View: Inventory → Assets → Asset Scores → Hosts → clic derecho sobre el host → Open Host Risk View.
- User Risk View: de forma equivalente, accedes al usuario desde las vistas de identidad/activos y abres su User Risk View. También puedes llegar a ella desde una issue relacionada con ese usuario.

**dashboard predefinidos por Cortex más típicos**

- **Agent Management:** estado de los Cortex XDR Agents, versiones, sistemas operativos y agentes desplegados.
- **Case Management:** casos principales, hosts implicados y asignación de casos entre analistas.
- **My Dashboard:** casos y métricas del analista conectado, como el *Mean Time To Resolve* (MTTR).
- **Security Admin:** visión global de los casos, incluyendo casos resueltos y *overdue*.
- **Security Manager:** visión general del SOC combinando información de casos y agentes.
- **Data Ingestion:** volumen de datos ingeridos, consumo de cuota y tasa de ingestión.
- **Automation Insights:** información sobre automatizaciones, ejecuciones y casos o issues cerrados automáticamente.

**¿Cuál es el número máximo de agentes que puede soportar un agente applet local Broker VM?**

10.000 agentes.

**What happens to network communications when an endpoint is isolated by Cortex XDR?**

Communication is restricted to Cortex XDR and allowed IP addresses/processes.

**After creating a new BIOC rule, how does Cortex XDR initially evaluate existing tenant data?**

It scans all historical data regardless of retention limits.

**¿De donde viene el nombre de una issue?**

El Issue Name viene del nombre de la regla o detector que generó la issue. The module that triggered the issue, i.e Behavioral Threat

**How do you create an issue Exception?** Right-click on an issue and select Create Issue Exception

**In which way can you create an Issue Exclusion** Create a new exclusion rule

### **Comunicación Cortex y Agente**

El funcionamiento es:

- Comunicación iniciada por el agente → heartbeat cada 5 minutos. El Cortex XDR Agent se comunica con Cortex XDR aproximadamente cada 5 minutos mediante un heartbeat. Ese heartbeat sirve para enviar estado del agente y comprobar/recibir información como políticas o versiones de contenido. Palo Alto documenta ese intervalo fijo de 5 minutos. Es por TCP/HTTPS (puerto 443) y también es aprovechada para enviar información sobre el agente (logs)
- Comunicación iniciada por Cortex → WebSocket seguro persistente. El agente mantiene una conexión WebSocket/WSS con Cortex XDR para que el servidor pueda enviar acciones prácticamente en tiempo real, por ejemplo aislamiento, kill process, quarantine, Live Terminal o file retrieval.

### **¿Qué cosas pueden generar alertas?**

- Las reglas (IOCs, BIOCs o correlation rules)
- Findings: un finding no es por sí mismo una issue, pero puede generar una issue si cumple una lógica determinada.
- Cortex XDR Agents
- Firewalls
- Analytics
- Integrations

**What are some of the specific security configurations that can be applied from parent tenants to child tenants?**

BIOC rules, alert policies, endpoint prevention profiles, and allow/block lists

**¿Qué es un schema?** Un schema es información descriptiva sobre un dataset, y se almacena en otro dataset. En esencia, un schema dataset proporciona la descripción de todos los campos (atributos) de un dataset determinado.

### **How are Dynamic Groups in Cortex populated?**

Automatically based on endpoint characteristics such as tags, hostname, and agent version.

**¿La protección Behavioral Threat Protection se encuentra en algún perfil?**

Sí. Behavioral Threat Protection (BTP) se configura dentro del Malware Security Profile.

**¿Qué hace cada perfil?**

- Perfil de Malware: analiza ficheros, DLLs, comportamientos (BTP) o inspecciona paquetes de red a nivel de protocolo.
- Exploit Protection: protege frente a técnicas de explotación como desbordamientos de buffers.

**Para analizar un fichero, ¿cuál es el orden de análisis?**

Hash Exception → Highly Trusted Signer → WildFire Verdict → Local Analysis.

**¿Qué son los perfiles de prevención?** Los Prevention Profiles contienen grupos de configuraciones que determinan qué EPMs se habilitan en determinados procesos y en el kernel, con el fin de prevenir que amenazas exploten vulnerabilidades o fallos de software.

**Recursos necesarios para instalar un agente.** En Windows una CPU Intel Pentium Dual Core o posterior / AMD equivalente, con SSE2, una RAM de 2 GB mínimo y una Disco de 5 GB mínimo; 20 GB recomendado.

**Recursos necesarios para instalar el Broker VM.**

For standard installation, use a minimum of a 4-core processor, 8 GB RAM, and 512 GB disk.